Supplementary Material for AgentISA: Formal Instruction Set Design for LLM Agents

Anonymous Author(s)

Abstract

This document is the supplementary material for *AgentISA: Formal Instruction Set Design for LLM Agents*. It contains the appendices referenced by the main paper: Appendix A (the Turing-completeness construction for Theorem 3.1), Appendix B (the type-safety proof for Theorem 3.2), Appendix C (the semantic-preservation lemmas and proofs of Theorems 3.4–3.6, with the scope remarks of C.7), Appendix D (component listings), Appendix E (formal sketches for future directions), Appendix F (artifact-regeneration commands), Appendix G (the priced-comparison protocol), Appendix H (the four-task characterization in full detail), and Appendix I (the extension-experiment protocols and detailed tables behind Sections 5.10–5.11). Section and theorem numbers follow the main paper.

A Proof of Theorem 3.1 (Turing Completeness)

A.1 Strategy

We prove that AgentISA can simulate any program in the WHILE language over the natural numbers. WHILE programs compute exactly the partial recursive functions—WHILE is Turing equivalent [43, 44]—so a sound simulation of WHILE by AgentISA establishes Turing completeness. The argument is a sufficiency (simulation) argument over instruction classes: the instruction classes of Definition 3.1 must jointly provide unbounded storage, sequential composition, conditional branching, and unbounded iteration. We show the control-flow class $\mathcal{C}$ and the state-management class $\mathcal{S}$ suffice; neither effectful class (the LLM-interaction class $\mathcal{L}$ nor the resource class $\mathcal{R}$) is needed, making the result conservative with respect to the full instruction set.

A.2 Source Language

Let WHILE be the language

$$P ::= x := x{+}1 \mid x := x{-}1 \mid x := y \mid \texttt{skip} \mid \texttt{if } x = 0 \texttt{ then } P_1 \texttt{ else } P_2 \mid \texttt{while } x \neq 0 \texttt{ do } P \mid P_1; P_2$$

over variables ranging over $\mathbb{N}$ (with predecessor cut off at 0), with the standard small-step semantics. Configurations are pairs $\langle P, \sigma \rangle$ with store $\sigma : Var \to \mathbb{N}$.

A.3 Encoding

(i) Store. A WHILE store σ is represented by the AgentISA state $s_\sigma \in \Sigma$, in the component managed by the state-management instructions. Each variable x occupies a dedicated cell; cells hold

unbounded naturals, so Σ provides an unbounded store—the crucial ingredient of the sufficiency argument.

(ii) Instruction semantics (as used here). The state-management component of Σ carries, alongside the variable cells, an **evaluation stack**; the five encoding instructions below operate on that stack. Five instructions suffice for the encoding, with the following operational meaning:

- `load` x: pushes the contents of the cell for x onto the evaluation stack;
- `store` x: pops the top of the evaluation stack into the cell for x;
- `update` (f): pops the top of the stack, applies the unary arithmetic function f, and pushes the result; `update`(-1) denotes the *truncated predecessor* $\lambda n.\ \max(n-1, 0)$, matching WHILE's cutoff semantics on $\mathbb{N}$;
- `branch` (b, Q_1, Q_2): pops the top of the stack; if predicate b holds, execute Q_1, else execute Q_2;
- `loop` (Q): execute Q; if its final `branch` returns to the loop head, repeat, else exit—that is, `loop` repeats its body for as long as the body's trailing test succeeds.

(iii) Statements. Each WHILE statement compiles to the AgentISA sequence below (notation defined after the table):

Table 1: Compilation of WHILE statements to AgentISA sequences ($\hat{P}$ is the compilation of P).

WHILE construct	AgentISA compilation
$x := y$	`load` y; `store` x
$x := x + 1$	`load` x; `update`$(+1)$; `store` x
$x := x - 1$	`load` x; `update`(-1); `store` x
`if` $x = 0$ `then` P_1 `else` P_2	`load` x; `branch`$(= 0?, \hat{P}_1, \hat{P}_2)$
`while` $x \neq 0$ `do` P	`loop`(`load` x; `branch`$(\neq 0?, \hat{P}, \epsilon)$)
$P_1; P_2$	$\hat{P}_1 \circ \hat{P}_2$ (sequential composition)

where $\hat{P}$ denotes the compilation of P. The no-op `skip` compiles to the identity instruction ϵ; the `while` row's trailing `branch` exits to ϵ when the guard fails, which is exactly the unfolding identity below.

A.4 Simulation Lemma

Lemma A.1. For every WHILE configuration $\langle P, \sigma \rangle$ and its encoded AgentISA instruction sequence $\hat{P}$ started in state s_σ:

(a) if $\langle P, \sigma \rangle \to \langle P', \sigma' \rangle$, then the AgentISA machine, started at configuration $(\hat{P}, s_\sigma)$ (remaining instruction sequence, Σ-state), reaches the configuration $(\hat{P}', s_{\sigma'})$—remaining sequence $\hat{P}'$ the encoding of the residual program P', state $s_{\sigma'}$ the encoding of σ'—in finitely many steps; and

(b) if $P =$ `skip` (terminal), then $\hat{P}$ terminates.

Proof. By structural induction on P:

- **Assignments** (base cases). Each compiled sequence performs exactly the corresponding store update; (a) holds by executing the two or three instructions in order, (b) trivially.

- **Conditional.** The `branch` instruction evaluates the zero test on the loaded value and transfers control to one of the compiled branch bodies. By the induction hypothesis applied to P_1 and P_2, each branch simulates its WHILE counterpart.
- **Loop.** The WHILE unfolding identity `while x ≠ 0 do P ≡ if x = 0 then skip else (P ; while x ≠ 0 do P)` is simulated by the `loop` instruction's iteration: each iteration loads x and executes the trailing `branch`, which dispatches to $\hat{P}$ when $x \neq 0$ and to ϵ (loop exit) when $x = 0$. By an outer induction on the number of executed iterations (in addition to the structural induction on P announced above—the induction hypothesis applied to P covers each body execution, and the trailing branch discharges the loop condition)—the k-th WHILE iteration is matched by the k-th `loop` iteration.
- **Sequence.** $\hat{P}_1 \circ \hat{P}_2$ executes $\hat{P}_1$ to completion, then $\hat{P}_2$; both simulations compose by the induction hypothesis.

The case arguments above are the substantive reduction steps; expanding each into a fully formal induction over the small-step rules (matching each WHILE step against the transition functions $\delta \in \Delta$ of the corresponding AgentISA instructions) is mechanical and follows the template of textbook simulation arguments [33, 43]. We mark those expansions as routine and leave them to the reader. ■

A.5 Conclusion

By Lemma A.1, every WHILE-computable function is AgentISA-computable. Since WHILE is Turing equivalent—a classical result of computability theory, with WHILE serving as the standard imperative model in semantics texts such as [43]—AgentISA is Turing complete. ■

A.6 Scope Remark

Turing completeness is a deliberately weak property—any language with unbounded storage, branching, and unbounded iteration attains it. Its role here is negative certification: the instruction set does not artificially constrain what agent programs can express, so no expressiveness objection can be raised against the architecture. The claim says nothing about efficiency, and it is established without the LLM-interaction class $\mathcal{L}$, so it is conservative with respect to the full instruction set of Definition 3.1.

B Proof of Theorem 3.2 (Type Safety)

We prove type safety for the pure core by progress and preservation, in the style of Pierce [36]. The proof runs over a **merged judgment system**: the term typing judgment $\vdash e : T$ for λ-terms, and the instruction judgment $\vdash i\ ok[pre(i), post(i)]$ for instructions and their compositions (both defined in B.1). Progress and preservation are each proved by induction over the derivations of this merged system: the λ-cases are the standard TAPL argument on term derivations; the instruction cases are discharged by the standing totality assumption and the per-operator premises of B.1(ii), with a joint runtime invariant threaded through the composition cases.

B.1 Core Calculus

Restrict attention to the pure core of the instruction language: expressions over base types, dependent Π/Σ pairs, recursion (μ), λ-abstraction and application (whose β-reduction rule appears

in Section 3.1.2), and the composition operators of Definition 3.6 applied to instructions carrying typed pre/post-conditions. The small-step relation $\to$ is generated by β-reduction, the context rule $E[e_1] \to E[e_1']$, and, for each instruction i, its **fixed per-instruction** transition function δ_i. Effectful instructions are treated in B.7.

The two judgment forms:

(i) **Term typing** $\vdash e : T$—the usual rules over base types, extended with the dependent Π/Σ introduction forms of Definition 3.2.

(ii) **Instruction typing** $\vdash i\ ok[pre(i), post(i)]$—read "$i$ is a well-contracted state transformer": $pre(i)$ and $post(i)$ are well-typed predicates over the state type, and the Definition 3.5 obligation $\forall s \vDash pre(i) : \delta_i(s) \vDash post(i)$ is discharged per instruction, offline, by the bounded verification of Section 3.3.2. Instructions are **not values and never stuck**: totality on precondition-satisfying states is a standing assumption—for each instruction i with its transition function δ_i, $\delta_i(s)$ is defined whenever $s \vDash pre(i)$.

Composition rules, for instructions i_1, i_2 carrying contracts:

$$\frac{\vdash i_1\ ok \qquad \vdash i_2\ ok \qquad post(i_1) \Rightarrow pre(i_2)}{\vdash i_1 \circ i_2\ ok[pre(i_1),\, post(i_2)]}$$

$$\frac{\vdash i_1\ ok \qquad \vdash i_2\ ok \qquad pre(i_1) \Leftrightarrow pre(i_2) \quad \text{disjoint state footprints}}{\vdash i_1 \parallel i_2\ ok[pre(i_1),\, post(i_1) \wedge post(i_2)]}$$

$$\frac{\vdash i_1\ ok \qquad \vdash i_2\ ok \qquad post(i_1)\big|_{\text{trigger}} \Rightarrow pre(i_2)}{\vdash i_1 \rhd i_2\ ok[pre(i_1),\, post(i_1) \wedge (\text{trigger} \Rightarrow post(i_2))]}$$

$$\frac{\vdash i\ ok \qquad \text{guard closed under } post(i) \qquad \text{guard} \Rightarrow pre(i)}{\vdash i^*\ ok[\text{guard},\, post(i)]}$$

Two premises deserve note. The $\rhd$ premise is evaluated **on the outcome of i_1 that triggers i_2**: i_1's execution can otherwise destroy $pre(i_2)$, so the implication the runtime invariant needs is $post(i_1)$ restricted to the triggering outcome, not $pre(i_1) \Rightarrow pre(i_2)$ on the initial state. The i^* premise adds guard $\Rightarrow pre(i)$ because the loop body re-enters: on every re-iteration the guard holds, so the body's precondition must hold whenever the loop has not terminated. These rules are generated from—do not add to—the semantics of Definition 3.6.

B.2 Canonical Forms

If $\vdash v : T$ and v is a value, then:

- $T \in \{Bool, Int, String, Float\}$ implies v is a literal of that type;
- $T = \Pi(x : T_1).T_2$ implies $v = \lambda x{:}T_1.\, e$ with $\Gamma, x{:}T_1 \vdash e : T_2$;
- $T = \Sigma(x : T_1).T_2$ implies $v = (v_1, v_2)$ with the components typed at the corresponding dependent types;
- $T = \mu X.T$ implies v is a rolled recursive value;
- no instruction or instruction composition is a value (B.1(ii)): instructions are contracted state transformers, not term-level values.

Proof. By inspection of the typing rules: each type has exactly one value form, and the syntax of values is defined per type, so the case analysis is exhaustive. This is the standard canonical-forms lemma [36]. ■

B.3 Lemma (Substitution)

If $\Gamma, x{:}T' \vdash e : T$ and $\vdash v : T'$, then $\Gamma \vdash e[x := v] : T$.

Proof. By induction on the derivation of $\Gamma, x{:}T' \vdash e : T$. The non-dependent cases are the case analysis of Pierce [36], which we do not duplicate. The dependent Π/Σ cases additionally require synchronizing the substitution into the second component's type ($T_2[x := v]$), as in the β-case of B.5—the standard dependent-type substitution bookkeeping, not covered by the non-dependent treatment in [36]. ■

B.4 Lemma (Progress)

If $\vdash e : T$, then e is a value or there exists e' with $e \to e'$.

Proof. By induction on the derivation. The λ-term cases:

- **Literals and** λ are values.
- **Application** $e_1\, e_2$ with $e_1 : \Pi(x : T_1).T_2$ and $e_2 : T_1$: by the induction hypothesis, either e_1 steps (then the context rule applies) or e_1 is a value, in which case B.2 gives $e_1 = \lambda x.\, e$; similarly e_2 either steps or is a value; when both are values, β-reduction applies. There is no stuck case, because the Π-type of e_1 and B.2 jointly guarantee a λ in value position.

The instruction cases admit no stuck state by construction. Instructions step by the standing totality assumption of B.1(ii) whenever their precondition holds. Compositions maintain the runtime invariant **each reached component's precondition holds**, per operator:

- **Sequencing** $\circ$**:** $pre(i_1)$ holds initially (the composition's contract); after i_1 executes, $post(i_1) \Rightarrow pre(i_2)$ re-establishes the invariant for i_2.
- **Parallel** $\|$**:** disjoint state footprints keep each component's precondition valid throughout the interleaving—neither component writes what the other's precondition or postcondition reads.
- **Guarded** $\triangleright$**:** on the triggering outcome of i_1, the strengthened premise $post(i_1)\big|_{\text{trigger}} \Rightarrow pre(i_2)$ re-establishes $pre(i_2)$; on the non-triggering outcome, i_2 does not run.
- **Iteration** i^***:** the body re-enters only while the guard holds, and the premise guard $\Rightarrow pre(i)$ makes each re-entry legitimate; when the guard fails, the loop is terminal.

The invariant is threaded jointly with B.5: each individual step preserves the invariant by the per-operator argument above. For iteration, either the guard is false (terminal) or the body's precondition holds by guard $\Rightarrow pre(i)$, and the body steps by the induction hypothesis on the component derivation. ■

B.5 Lemma (Preservation)

If $\vdash e : T$ and $e \to e'$, then $\vdash e' : T$.

Proof. By induction on the derivation, with case analysis on the step:

- β**:** $(\lambda x.\, e)\, v \to e[x := v]$. The application rule types the redex at $T_2[x := v]$; by B.3, the substituted term $e[x := v]$ types at $T_2[x := v]$, which is exactly the type the application rule assigned (the dependent-type bookkeeping; when x does not occur in T_2 this degenerates to the usual $T_2 = T$).
- **Context:** $E[e_1] \to E[e_1']$ with $e_1 \to e_1'$. By the induction hypothesis $e_1' : T_1$; an induction on the structure of E re-types $E[e_1']$ at the original type.
- **Instruction transitions:** the instruction's *ok* judgment (B.1(ii)) certifies that δ_i establishes $post(i)$ from any $pre(i)$-satisfying state (discharged offline by the bounded verification of Section 3.3.2), so the reduct re-types at the postcondition.
- **Composition operators:** each operator's step is typed by its B.1(ii) rule; the reduct's contract follows from the operator's stated conclusion (for $\circ$, $post(i_2)$; for $\|$, the conjunction; for $\rhd$, the trigger-restricted conjunction; for i^*, $post(i)$), by the induction hypothesis on the component derivations.

The remaining instruction-class cases repeat this pattern with class-specific invariants; they are routine and omitted (the template is the standard preservation argument for the corresponding constructs [36]). ■

B.6 Theorem 3.2

By B.4, a well-typed closed term is a value or takes a step; by B.5, every step preserves typing. Hence no well-typed term reaches a stuck state: well-typed programs in the pure core cannot go wrong. ■

B.7 Scope Remarks

Two caveats state the theorem's boundaries honestly. First, B.4's appeal to totality of transition functions on precondition-satisfying states is an assumption about instruction *implementations*, not something the type system alone discharges. The per-instruction mechanism intended to discharge it is the bounded verification of Section 3.3.2, which is now shipped code: the checker mechanically discharges the obligation for pure instructions on the state spaces it can enumerate—exhaustively for small enumerable domains, by deterministic bounded sampling beyond—while side-effectful instructions and state spaces beyond the enumeration bound remain covered by hand argument and unit tests rather than by the mechanical check. The theorem is conditional on the obligation being (or becoming) true of each instruction, to the extent stated. Second, effectful instructions (LLM calls, resource allocation) can fail at runtime for reasons outside the type system (network errors, exhausted capacity); Theorem 3.2 covers the pure core and effectful instructions with checkable preconditions. Runtime failure handling for the remaining cases is the responsibility of the fallback machinery of Section 5.6, not of types.

C Proofs of Theorems 3.4–3.6

C.1 Denotational Setup

Fix the state space Σ (Definition 3.1's state space—not the dependent-type constructor Σ of Definition 3.2). For a pure instruction i, let $[\![i]\!] : \Sigma \rightharpoonup \Sigma$ denote its denotation, a partial function total on states satisfying $pre(i)$. Program denotation is defined by structural recursion on the composition algebra of Definition 3.6:

- $[\![i]\!]$ for atomic instructions (given);
- $[\![p_1 \circ p_2]\!] = [\![p_2]\!] \circ [\![p_1]\!]$ (note the order: p_1 executes first);
- $[\![p_1 \parallel p_2]\!](\sigma) = [\![p_2]\!]([\![p_1]\!](\sigma))$ for pure p_1, p_2 with disjoint state footprints (as the $\parallel$-typing rule requires)—the order is immaterial because updates to disjoint cells commute on their domains;
- $[\![p_1 \rhd p_2]\!](\sigma) = [\![p_2]\!]([\![p_1]\!](\sigma))$ when p_1's outcome at σ is the trigger, and $[\![p_1]\!](\sigma)$ otherwise (the denotation is defined on $\Sigma +$ outcome, with p_2 applied conditionally on the trigger tag);
- $[\![p^*]\!]$ is the least fixed point of the corresponding functional.

Write $p \equiv q$ iff $[\![p]\!] = [\![q]\!]$ on all states where both are defined.

C.2 Lemma (Atomic Rewrite Soundness)

Each rewrite rule implemented by the optimizer (Section 3.4.4) preserves denotation.

(i) Fusion. $i_1 \circ i_2 \to i_{12}$, with $pre(i_{12}) = pre(i_1)$, $post(i_{12}) = post(i_2)$, and i_{12}'s body stipulated to compute $[\![i_2]\!] \circ [\![i_1]\!]$ (intermediate bindings named, so the composition is explicit). The denotational claim is then an instance of the definition of $\circ$ (C.1): a body computing $[\![i_2]\!] \circ [\![i_1]\!]$ denotes exactly $[\![i_1 \circ i_2]\!]$ (the associativity of function composition is what fuses *three or more* consecutive instructions). **Two honest boundaries.** First, this is an equality *of the rewrite algebra*: it holds of the fused form whose body computes the composite. The shipped *formal compiler* (the static path of the Section 5.3 benchmarks) produces a formal fused *node* (merged metadata and dependencies) without generating an executable composite body—code generation for fused nodes is not implemented—so what those benchmarks measure is the structural fusion of the formal stream, and the executable-fusion claim would additionally require the codegen step. The *Instruction-level* merge path (the dynamic optimizer of Sections 4.2 and 5.4) does carry an executable body—the `CompositeInstruction` replay construction—and its differential property (composite execution equals constituent-sequence execution; the fusion gate refuses producer–consumer pairs) is pinned by unit tests and exercised by the trace-replay identity of Section 5.4 (5/5 seeds). A fused *single* executable body distinct from replay remains specified. Second, the compiler applies fusion only when output types of i_1 match input types of i_2 and both members are pure, since denotational equality of the fused form presupposes order-insensitivity of intermediate effects, and the shipped gate achieves it via purity (the pure or order-insensitive alternative is specified, not implemented).

(ii) Dead-code elimination. Instruction i is removed from p when $Def(i) \cap Live_{out}(i) = \emptyset$, with liveness the standard fixpoint of $Live_{in}(i) = Use(i) \cup (Live_{out}(i) - Def(i))$ [33]. Claim: if i is dead and side-effect-free, then $p \equiv p \setminus \{i\}$. Proof sketch: by induction on the suffix program following i, its denotation depends only on the live variables at i's position; i writes none of them, so removing i cannot change any value subsequently read. The side-effect-freeness gate matches the purity restriction of Definition 3.6 (Property 2 and the Remark following it)—instructions declared `side_effects=True` are never eliminated.

(iii) Peephole combining. Each rule $(s_1, \ldots, s_k) \to s'$ carries the verification obligation $\forall \sigma : [\![s_1 \circ \ldots \circ s_k]\!](\sigma) = [\![s']\!](\sigma)$, to be discharged at rule-definition time (Section 4.1.3). Candidate instances, *illustrative only—the shipped optimizer registers none of them*: `LOAD A; STORE B` composes to `MOVE A→B` (degenerate to the identity when A = B, since the value read is the value written; the general rule of Section 3.4.4); consecutive `QUERY` instructions with identical (instruction, params) pairs collapse to one query whose result is reused (the batch-QUERY rule of Section 3.4.4, specialized to identical pairs) (an obligation about a *compile-time* rewrite; the caching measured in Section 5.2 is a different, *runtime* mechanism—a result cache consulted at execution time—and its numbers do

not evidence this rule); consecutive `CONTEXT_UPDATE`s compose to a single update by associativity of the update function.

Proof. Each case reduces to the construction of the rewritten instruction and the definition of liveness. The inner inductions—on the pair of bodies for (i), on suffix programs for (ii)—are routine and left to the reader. ■

Distributivity necessity arguments (moved verbatim from the main text's Section 3.4). *Both premises are necessary, and neither follows from the other or from purity.* Purity not implying idempotence is the less obvious of the two failures: a pure `update(+1)` applied twice increments twice (purity bars *external* side effects, not writes to declared cells). Taking i_1 as such an update and $i_2 = i_3 = \epsilon$ (the identity, empty footprint, so (b) holds vacuously), the two sides execute i_1 once and twice—from $x = 0$ they yield $x = 1$ and $x = 2$; only premise (a) rules this out. Conversely, without (b), a write of i_2 into a cell i_1 reads changes i_1's second execution even when i_1 is idempotent.

C.3 Lemma (Compositionality)

Let T be a program transform with $[\![T(p)]\!] = [\![p]\!]$ for all p. Then for any context $C[\cdot]$ built from the composition operators, $[\![C[T(p)]]\!] = [\![C[p]]\!]$.

Proof. By structural induction on C. Each operator's denotation (C.1) is defined compositionally in its constituents' denotations, so substituting a denotationally equal constituent leaves the whole unchanged. The parallel case uses the disjoint-footprint hypothesis of the $\|$-typing rule (updates to disjoint cells commute). The iteration case uses the defining property of least fixed points: if the functionals induced by p and $T(p)$ are equal, then all finite approximants are equal (by the induction hypothesis), so their suprema—the least fixed points—are equal. ■

C.4 Theorem 3.4 (Optimization Soundness)

Proof. Read all equalities below as the partial-equivalence $\equiv$ of C.1 (equality on all states where both sides are defined), since the DCE rule preserves denotation only on the defined domain and can strictly enlarge it. The compiler applies a finite sequence of rewrites $O = R_k \circ \ldots \circ R_1$ (finiteness follows from compilation terminating; each pass is a bounded traversal). By induction on k: $k = 0$ is trivial. For the step, write $O = R \circ O'$. By the induction hypothesis $[\![O'(p)]\!] \equiv [\![p]\!]$. Two routes, by rule kind. **Fusion and peephole** are local subprogram rewrites: the atomic-rewrite lemma (C.2) preserves the denotation of the subprogram it rewrites, and the compositionality lemma (C.3) lifts the preservation through any surrounding composition context. **Dead-code elimination** is not local—liveness is computed over the whole surrounding program—so it is handled at pass level directly by C.2(ii)'s whole-program argument ($p \equiv p \setminus \{i\}$), not through the lifting template. Hence $[\![R(O'(p))]\!] \equiv [\![O'(p)]\!] \equiv [\![p]\!]$. ■

C.5 Theorem 3.5 (Pipeline Correctness)

Let s be a valid instruction sequence and S a schedule satisfying:

- **(S1)** every instruction is dispatched exactly once;
- **(S2)** for every dependency $(i_1, i_2) \in \mathit{Deps}$: $\mathit{start}(i_2) \geq \mathit{end}(i_1)$ (the constraint of Section 3.4.3);
- **(S3)** instructions sharing a parallel dispatch group are pairwise non-interfering (no data hazard, no resource conflict);

- **(S4)** commit is in program order.

Then executing s under S yields the same final state as sequential execution: $[\![s]\!]_S = [\![s]\!]_{seq}$.

Proof. Let $g_1, \ldots, g_m$ be the dispatch groups of S in commit order. By S4, groups commit in program order and group commits do not interleave, so the union $\bigcup_{j \le t} g_j$ of committed groups is always a program-order prefix—"executing all instructions in those groups" below means executing that prefix sequentially. We show the invariant

$$I(t) : \text{after committing } g_1, \ldots, g_t,$$
$$\text{the architectural state equals the sequential state after}$$
$$\text{executing all instructions in those groups.}$$

- **Base** ($t = 0$): both sides are the initial state.
- **Step**: assume $I(t)$. Every instruction in g_{t+1} has its dependencies in earlier groups (by S2, since it is scheduled after them), which by $I(t)$ have completed and committed. Within the group, S3 gives pairwise non-interference: no instruction reads another group-member's output, and no two write the same location or resource. Hence the group's updates commute—for pure updates this is the diamond property, and for effectful instructions, S3's exclusion of shared resources makes their effects independent, so the combined effect is unique regardless of interleaving. The state after g_{t+1} is therefore independent of the group's internal interleaving and equals the sequential execution of the same instructions from the state at $I(t)$.
- By induction, $I(m)$ holds; by S1 no instruction was skipped or duplicated.

This completes the induction over all dispatch groups, hence the proof. ■

Corollary (dispatch folding). The ADVANCED and AGGRESSIVE levels fold instructions into shared dispatches (Section 5.3). A folded group satisfies S3 by construction: the grouping check admits only side-effect-free instructions whose dependencies and declared outputs are pairwise disjoint, so the group's members act on disjoint state components and their updates commute (Definition 3.6, Property 2). By Theorem 3.5 the fold changes the *dispatch count* but neither the semantics nor—consistent with the measurement of Section 5.3—the effective instruction count.

C.6 Theorem 3.6 (Pass-Pipeline Soundness)

Every optimization in the pass pipeline is a finite composition of rewrites covered by the atomic-rewrite lemma (C.2) or by the dispatch-folding corollary of Theorem 3.5 (C.5)—the grouping pass is a fold, not a local rewrite—and the argument of C.4 applies verbatim to each rewrite layer. ■

C.7 Scope Remark

Theorems 3.4–3.6 are *conditional*: they hold of transformations whose verification obligations (Sections 4.1.3 and 4.3.1) are discharged. As stated there, those obligations are **specified but not yet mechanically discharged**—the shipped optimizer registers no peephole rules (the peephole obligation is dormant, and the fusion/DCE/grouping passes are argued directly in C.2 and C.5 rather than through the symbolic-execution checker), and the symbolic-execution equivalence checker itself is a specification (Section 4.3.1). Where an obligation is discharged only on bounded state spaces (Section 3.3.2), the guarantee is correspondingly bounded: rewrites verified on the explored subset are sound there. No such rewrite is registered in the default configuration; when one is—through the approval-gated promotion path of the main paper's Section 3.3.1, or future rule authoring—its

application must be restricted to the explored subset (that restriction remains a dormant obligation, not enforced by the shipped gate). What the shipped implementation does provide is empirical semantics-preservation at task level: the system-level demonstration of Section 5.8 re-executes an optimized stream and checks byte-identical output.

Observation model. The theorems' equality is *partial equivalence* (C.1: $[\![p]\!] = [\![q]\!]$ on states where both are defined), and the observation model should be read accordingly. **Observed:** the final state of declared state cells, the declared outputs (the instruction's product keys), and—for theorems covering effectful instructions (C.5)—the externally visible effects, in program order. **Deliberately not observed:** the *domain of definition* itself. Because equivalence is only partial, a rewrite may extend definedness—DCE is the canonical example (C.4): deleting an instruction that would have failed can turn a previously undefined run into a defined one. If program failure or exception behavior were part of the observation model, this would violate preservation; it is therefore stated explicitly that failure behavior is outside what Theorems 3.4–3.6 preserve, and any deployment for which failures are observable (e.g., a pipeline contract that treats exceptions as data) needs a strengthened equivalence and must re-derive the theorems under it. Internal traces—the sequence of instructions executed—are likewise not observed; only the net transformation is.

Three status levels. Each mechanism in this paper falls into exactly one of three levels, which we keep separate throughout: *(i) specified*—the symbolic-execution equivalence checker, the pipeline scheduler's cycle-accurate extension, block-granular trace formation (Sections 4.1.2 and 4.2.2); *(ii) implemented with checks*—the optimizer passes (fusion, DCE, grouping) with their pass-level arguments in C.2/C.5, the runtime monitors, and differential unit tests pinning the fusion property this appendix argues (fused-composite execution equals constituent-sequence execution; the fusion gate refuses producer–consumer pairs; `tests/core/isa/test_fusion_differential.py`); *(iii) empirically validated*—trace-level replay identity on the mock (Section 5.4: optimized traces re-executed, final states identical in 5/5 seeds), task-level re-execution with byte-identical outputs (Section 5.8), and the caching/compilation measurements of Section 5 (including the exact-response-cache parity baseline of Section 5.2). A statement at one level is never used to imply a guarantee at a stronger level.

D Component Listings

This appendix carries the algorithm listings moved from the main paper to respect the page limit; each listing keeps its original framing. Two listings formerly specified-only now describe shipped opt-in components, updated in place: D.1 (the MATCH-TREE selector: `selection_strategy="dp"` in `agentflow.core.isa.instruction_selector`, default heuristic) and D.4 (the EMA hot-path detector: `detection_mode="ema"` in `agentflow.core.isa.instruction_optimization`). The remainder stay as labeled—specified-only (D.2, D.3, D.5–D.7, D.9) or shipped (D.8, D.10, and D.11, the bounded-checker listing relocated from the main paper's Section 3.3.2). The default selection step (the heuristic name-overlap matcher) remains in the source module `agentflow.core.isa.instruction_selector`.

D.1 Tree-Pattern Instruction Selection (MATCH-TREE)

Implementation status: shipped as the opt-in selector of `agentflow.core.isa.instruction_selector` (`selection_strategy="dp"`, engaging only on inputs carrying an `ir_tree`; the default remains the heuristic matcher). `_match_tree` follows the listing's recurrence—candidate-pattern cost plus the optimal subtrees of unspanned children, memoized top-down—and exposes `(costs,`

`selected)` through `TreeSelectionResult`; matching is root-op equality (case-insensitive) with each catalogue pattern's `params["spanned_child_ops"]` spanning whole child subtrees, and the covering is read root-first, skipping spanned subtrees.

Input: IR tree T, pattern set P; output: optimal pattern selection S.

```
Input: IR tree T, Pattern set P
Output: Optimal pattern selection S

function MATCH-TREE(T, P)
    costs[n] ← ∞ for all nodes n in T
    Let selected[n] = ∅ for all nodes n in T

    function MATCH-SUBTREE(node)
        if costs[node] ≠ ∞ then
            return costs[node]
        end if

        min_cost ← ∞
        best_pat ← null

        for each pattern p in P do
            if MATCHES(p, node) then
                curr_cost ← PATTERN-COST(p)
                for each child c of node not covered by p do
                    curr_cost ← curr_cost + MATCH-SUBTREE(c)
                end for

                if curr_cost < min_cost then
                    min_cost ← curr_cost
                    best_pat ← p
                end if
            end if
        end for

        costs[node] ← min_cost
        selected[node] ← best_pat
        return min_cost
    end function

    MATCH-SUBTREE(root(T))
    return (costs, selected)
end function
```

Theorem 4.1 (Selection Optimality, shipped opt-in selector): given a set of context-local patterns P and an IR tree T, the MATCH-TREE algorithm computes the minimal covering cost; the covering itself is read off the `selected` table afterwards, by walking from the root and skipping subtrees spanned by the chosen pattern.

Proof: By induction on tree height, assuming a covering of T exists (if no pattern matches a node

its cost is ∞ and no covering exists):

1. Base case (a leaf has no children): the recurrence degenerates to a plain argmin over the patterns matching the leaf, which is optimal for a single node.
2. Inductive step: For a node v with children $c_1, \ldots, c_k$:

$$\text{cost}(v) = \min_{p \in P} \left\{ p.\text{cost} + \sum_{\substack{c \in \text{children}(v) \\ c \notin \text{covered}(p)}} \text{cost}(c) \right\}$$

   By the induction hypothesis, each $\text{cost}(c)$ is the minimal covering cost of the corresponding subtree; the recurrence takes each candidate pattern's cost plus the (already optimal) costs of the children the candidate does not span, so the minimum over candidate patterns is the minimal covering cost at v—the minimum over disjoint pattern choices is optimal.

The recurrence sums only over children not covered by the candidate pattern p—summing over all children would double-count the cost of subtrees the pattern spans. Section 3.4.3's per-node argmin is therefore exact for single-node patterns; multi-node patterns require this covering recurrence.

The recurrence requires that a pattern's feasibility at a node not depend on the covers chosen for other subtrees; context constraints that couple sibling subtrees break this optimal-substructure property, which is why the statement restricts to context-local patterns. This theorem is a property of the shipped opt-in selector, not of the default heuristic; no optimality claim is made for the default path.

D.2 Specified Cycle-Accurate Pipeline Scheduler (SCHEDULE-PIPELINE)

Input: instruction sequence I; output: schedule S with cycle assignments.

```
Input: Instruction sequence I
Output: Schedule S with cycle assignments

function SCHEDULE-PIPELINE(I)
    D ← BUILD-DEPENDENCY-GRAPH(I)
    S ← empty schedule
    R ← priority queue of ready instructions

    while D not empty do
        c ← CURRENT-CYCLE()

        // Find newly ready instructions
        N ← GET-READY-NODES(D, c)
        ADD-TO-QUEUE(R, N)

        // Schedule instructions for current cycle
        while R not empty and HAS-RESOURCES(c) do
            i ← POP-HIGHEST-PRIORITY(R)
            slot ← ALLOCATE-RESOURCES(i, c)
            ADD-TO-SCHEDULE(S, i, slot)
```

```
            UPDATE-DEPENDENCIES(D, i)
        end while
    end while

    VERIFY-SCHEDULE(S)
    return S
end function
```

D.3 Specified Dynamic Pipeline Controller (PipelineController)

```
class PipelineController:
    def __init__(self):
        self.hazard_detector = HazardDetector()
        self.resource_monitor = ResourceMonitor()
        self.performance_tracker = PerformanceTracker()

    async def execute_pipeline(self,
                               instruction_stream: AsyncIterator[Instruction]
                              ) -> AsyncIterator[Result]:
        async for inst in instruction_stream:
            # Hazard detection
            while self.hazard_detector.has_hazard(inst):
                await self.hazard_detector.wait_resolution()

            # Resource allocation
            resources = await self.resource_monitor.allocate(inst)

            try:
                # Execution
                result = await self.execute_instruction(inst, resources)
                # Performance tracking
                self.performance_tracker.record(inst, result)
                yield result
            finally:
                self.resource_monitor.release(resources)
```

D.4 Hot-Path Detector (DETECT-HOT-PATH)

Input: execution path P, threshold θ, EMA decay γ ($\gamma = 2/(\text{w}+1)$ for window w; γ is unrelated to the α stability bound of the main paper's Section 3.4.4); output: Boolean indicating if P is hot.

```
Input: Execution path P, Threshold θ, EMA decay γ (γ = 2/(w+1) for window w; γ is unrelated to t
Output: Boolean indicating if P is hot

function DETECT-HOT-PATH(P, θ, γ)
    Let counter be an exponential moving average with decay γ

    function UPDATE-COUNTERS(path)
        h ← COMPUTE-PATH-HASH(path)
```

```
        counter[h] ← counter[h] × (1 - γ) + 1

        if IS-PATH-HOT(path) then
            TRIGGER-OPTIMIZATION(path)
        end if
    end function

    function IS-PATH-HOT(path)
        h ← COMPUTE-PATH-HASH(path)
        freq ← counter[h] / Σ_{h'} counter[h']   // recent execution share, not cumulative total
        stab ← COMPUTE-STABILITY(path)
        return freq > θ and stab > STABILITY-THRESHOLD
    end function

    UPDATE-COUNTERS(P)
    return IS-PATH-HOT(P)
end function
```

Implementation status: shipped as the opt-in `detection_mode="ema"` of `agentflow.core.isa.instruction_optimization`, wired through `AgentISAConfig.hot_path_detection/ema_window/ema_threshold/max_tracked_paths` (defaults exact / 10 / 0.5 / 1,000). Implementation-fidelity notes: the detector remains exact-path-keyed (no similarity merging); a path's EMA counter must exceed a single fresh observation before its share is evaluated (a cold-start debounce, so the first recorded path cannot self-trigger at share 1.0); the share denominator sums the counters of LRU-tracked paths, and eviction shrinks it; the stability gate reuses the shipped `>=` predicate rather than the listing's strict `>`; windows below 2 are rejected ($\gamma = 1$ would be memoryless). Trace formation (D.5) remains specified, not shipped.

D.5 Specified Trace Formation (BUILD-TRACE)

Input: hot path H; output: optimized trace T. Trace validity is *defined* by the conjunction of three properties (H the loop header of the region, tail the last block of the trace); VERIFY-TRACE-PROPERTIES checks them:

$$\begin{aligned} \text{Valid(trace)} \iff & \text{ Entry(trace) dominated by } H \\ & \wedge \text{ Exit(trace) post-dominates tail} \\ & \wedge \text{ NoSideExit(trace) except tail} \end{aligned}$$

```
Input: Hot path H
Output: Optimized trace T

function BUILD-TRACE(H)
    T ← empty trace

    function SHOULD-INCLUDE(block)
        return EXECUTION-FREQUENCY(block) > THRESHOLD and
               PREDICTABLE-BRANCHES(block)
    end function
```

```
    function SHOULD-SPLIT(block)
        return HAS-SIDE-EXIT(block) or
               EXECUTION-FREQUENCY(block) < MIN-FREQ
    end function

    for each block B in H do
        if SHOULD-INCLUDE(B) then
            APPEND-TO-TRACE(T, B)
        else
            break   // any block not selected terminates the trace
        end if
    end for

    VERIFY-TRACE-PROPERTIES(T)
    return T
end function
```

D.6 Specified Symbolic-Equivalence Checker (VERIFY-TRANSFORMATION)

Input: original CFG O, transformed CFG T; output: Boolean indicating semantic equivalence.

```
Input: Original CFG O, Transformed CFG T
Output: Boolean indicating semantic equivalence

function VERIFY-TRANSFORMATION(O, T)
    // Build symbolic execution paths
    P₁ ← SYMBOLIC-EXECUTE(O)
    P₂ ← SYMBOLIC-EXECUTE(T)

    function VERIFY-EQUIVALENCE(path₁, path₂)
        // σ ranges over the symbolic states Σ explored on both paths
        for each symbolic state σ ∈ Σ do
            if EVALUATE(path₁, σ) ≠ EVALUATE(path₂, σ) then
                return false
            end if
        end for
        return true
    end function

    // Paths must pair one-to-one; a length mismatch means the
    // transformation added or removed a symbolic path — reject.
    if LENGTH(P₁) ≠ LENGTH(P₂) then
        return false
    end if

    // Verify all path pairs, paired by the transformation's path map
    for each (p₁, p₂) in PATH-MAP(P₁, P₂) do
```

```
        if not VERIFY-EQUIVALENCE(p₁, p₂) then
            return false
        end if
    end for

    return true
end function
```

D.7 Specified Performance Monitor

```
class PerformanceMonitor:
    def monitor_execution(self,
                          instruction: Instruction,
                          context: Context) -> Metrics:
        start = time.perf_counter_ns()
        result = self.execute(instruction, context)
        end = time.perf_counter_ns()

        return Metrics(
            latency=(end - start),
            resource_usage=self.measure_resources()
        )
```

D.8 Shipped Peephole Optimizer (Empty Pattern Table)

Status: shipped in `agentflow.core.isa.auto_promotion` (flag `auto_approve`, default `False`; with `min_support = 2` and `min_success_rate = 0.7`). The optimizer itself is the `optimize` loop of `agentflow.core.isa.instruction_optimization`, whose pattern table is populated through `add_peephole_pattern`; the table ships empty by design, because mined composites reach it only through the audited `AutoPromotionEngine` gate—candidates must clear the support and all-steps-succeeded thresholds, and are then proposed for human approval rather than applied, unless `auto_approve` is enabled. The Section 5.11 replay experiment does *not* run behind this gate: its composite is mined and promoted through the learner's own threshold-validated path (`InstructionLearner.learn_from_traces` → `create_instruction`, Appendix I.2.6), leaving the optimizer's rule set untouched. The approval-gated engine itself ships and is exercised by its unit suite (`tests/core/isa/test_auto_promotion.py`), not by any Section 5 experiment.

```
class PeepholeOptimizer:
    def optimize(self, instructions: List[Instruction]) -> List[Instruction]:
        patterns = self.load_patterns()
        optimized = []
        i = 0
        while i < len(instructions):
            matched = False
            for pattern in patterns:
                if pattern.matches(instructions[i:i+pattern.length]):
                    optimized.extend(pattern.replace(instructions[i:i+pattern.length]))
                    i += pattern.length
                    matched = True
```

```
                break
            if not matched:
                optimized.append(instructions[i])
                i += 1
        return optimized
```

D.9 Specified Context-Aware Optimization Sketch (OPTIMIZE-CONTEXT)

Programmatic sketch from the main paper's Section 6.3.2, item 2 (future work; the performance model and the variant generator are future work, not implemented machinery):

```
function OPTIMIZE-CONTEXT(I, C)
    Let M be a context-performance model, R the rewrite rules
    for each context c in C do
        if PREDICT-PERFORMANCE(M, I, c) < THRESHOLD then
            candidates ← GENERATE-VARIANTS(I, R)
            I ← SELECT-BEST(candidates, c)
            UPDATE-MODEL(M, I, c)
        end if
    end for
    return I
end function
```

D.10 Shipped Coordination Flow and Fusion Pass Listings

Status: shipped in `agentflow.core.isa.coordination` (flag `strict`, default `False`)—`SecurityMonitor(config, strict: bool = False)` runs the policy checks in the fail-open default posture, and the opt-in fail-closed mode is reached through `AgentCoordinator(config, security_strict: bool = False)`; both default to fail-open, and the paired posture matrix exercising the difference is documented in Appendix I.2.4.

The coordination flow implemented by `AgentCoordinator` (authenticate, authorize, select by capability, plan stages with fallback, execute, record history), referenced by the main paper's Section 5.6:

```
Input: Agent set A, Task list τs
Output: Coordinated solutions S

function COORDINATE-AGENTS(A, τs)
    for each task T in τs do
        AUTHENTICATE(task issuer)              // SecurityMonitor
        AUTHORIZE(task, role policy)           // SecurityMonitor
        ag ← SELECT-BY-CAPABILITY(A, T)        // CapabilityProfile (Section 3.5)
        plan ← BUILD-EXECUTION-PLAN(T)         // stages + coordination rules
        task-outcome ← success
        for each stage in plan do
            stage-outcome ← EXECUTE-STAGE(stage, ag)
            if stage-outcome failed and fallback(stage) exists then
                stage-outcome ← EXECUTE-FALLBACK(stage)
            end if
```

```
            if stage-outcome failed then
                task-outcome ← failed
                break
            end if
        end for
        RECORD-HISTORY(T, task-outcome)
    end for
end function
```

The shipped instruction-combining (fusion) pass of the main paper's Section 4.1.3, also in the source module `agentflow.core.isa.instruction_optimization`:

```python
def combine_instructions(self, insts: List[Instruction]) -> List[Instruction]:
    combined = []
    current_group = []

    for inst in insts:
        if self.can_combine(current_group, inst):
            current_group.append(inst)
        else:
            if current_group:
                combined.append(self.merge_group(current_group))
            current_group = [inst]

    if current_group:
        combined.append(self.merge_group(current_group))
    return self.verify_combinations(combined)
```

The illustrative candidate rule set referenced by the main paper's Section 3.4.4 (textbook mnemonics, explicitly *not* shipped opcode names; specified, not registered—the shipped write path is the audited promotion gate of the main paper's Sections 3.3.1 and 4.1.3):

```
Combine Rules {
  // LOAD/STORE/MOVE are textbook mnemonics, illustrative
  // only; they are not shipped opcode names
  LOAD followed by STORE → MOVE
  Multiple CONTEXT_UPDATE → Single UPDATE
  Sequential QUERY → Batch QUERY
}
```

D.11 Shipped Bounded-Checker Listing (verify_instruction)

Relocated from the main paper's Section 3.3.2 to respect the page limit; the shipped module is `agentflow.core.isa.bounded_checker`.

```python
def verify_instruction(instruction, pre_condition, post_condition):
    if instruction.side_effects: return skipped(instruction)  # never executed
    state_space = generate_state_space(instruction)  # bounded: exhaustive or seeded, cap 256
    for state in state_space.states:
        if pre_condition(state):
```

```
            next_state = execute(instruction, state)
            check(progress, preservation, post_condition, next_state)  # structured violation re
    return verdict  # checked_states, exhaustive, violations by kind, passed
```

E Formal Sketches for Future Directions

Sketches referenced by the main paper (Sections 3.4 and 6.3); all are future work or specified-only, with no current implementation.

E.1 Stated Sketches

Compositional reasoning (Section 6.3.1, item 2): a parallel-composition rule

$$\frac{\{P_1\}S_1\{Q_1\} \quad \{P_2\}S_2\{Q_2\} \quad \text{disjoint}(S_1, S_2)}{\{P_1 \wedge P_2\}S_1 \parallel S_2\{Q_1 \wedge Q_2\}}$$

whose disjointness side condition mirrors the $\parallel$-typing rule of the main paper's Definition 3.6.

Capability type theory (Section 6.3.1, item 3):

```
Type Capability = {
  Basic: {Perception, Action, Memory}
  Composite: Basic → Basic
  Emergent: P(Composite)
}
```

with typing rules

$$\frac{\Gamma \vdash a : \text{cap}_1 \quad \Gamma \vdash b : \text{cap}_2}{\Gamma \vdash a \circ b : \text{cap}_1 \otimes \text{cap}_2}$$

Resource consumption model (Section 6.3.3, item 2): $R(i) = \alpha C_{cpu}(i) + \beta C_{mem}(i) + \gamma C_{io}(i)$ with platform-specific coefficients.

Instruction scheduling formulation (Section 3.4.3, the specified, not shipped, scheduler's objective): minimize the total stall $\sum_i \text{delay}(i)$ subject to $\text{start}(i_2) \geq \text{end}(i_1)$ for every dependency pair $(i_1, i_2) \in \text{Deps}$ and $\sum_{i \in \text{active}(t)} \text{res}(i) \leq \text{Capacity}$ at every time t.

Performance model (Section 3.4.5, specified; the benchmarks of Section 5 measure latency and token counts directly and do not evaluate it):

$$\text{Perf}(p) = \sum_{i \in p} (\text{base_cost}(i) + \text{overhead}(i) - \text{optimization_gain}(i))$$

where optimization gain is

$$\text{optimization_gain}(i) = \sum_{o \in \text{Opts}} \text{benefit}(o, i) \times \text{application_rate}(o, i).$$

Information flow control (Section 6.3.6, item 1):

$$\text{Secure}(P) \iff \forall s_1, s_2 : L(s_1) = L(s_2) \implies L([\![P]\!]s_1) = L([\![P]\!]s_2)$$

where L assigns security levels to states and outputs (classical noninterference, terminating runs only).

F Artifact Regeneration Commands

Exact command lines mirrored from the repository's `REPRODUCING.md`, referenced by the main paper's Artifact Availability section.

F.1 Section 5.2–5.5 microbenchmarks

```
./venv/bin/python examples/benchmarks/isa_microbench.py \
    --experiments cache compiler dynamic ablation --seeds 5
```

F.2 Section 5.7 real-gateway cache run

```
./venv/bin/python examples/benchmarks/isa_real_llm_bench.py \
    --backend gateway \
    --tasks 200 --seeds 1 --zipf 0.5 1.0 1.5 --max-tokens 64 \
    --output examples/benchmarks/isa_real_llm_results.json
```

(requires a reachable gateway; the base URL, API key, and CA certificate are passed per deployment). Any Anthropic-protocol gateway works—configure `ANTHROPIC_BASE_URL` and `ANTHROPIC_AUTH_TOKEN`; the paper's run used a local alias for a hosted GLM reasoning model:

```
./venv/bin/python examples/benchmarks/isa_real_llm_bench.py \
    --backend gateway \
    --tasks 200 --seeds 1 --zipf 0.5 1.0 1.5 --max-tokens 64
```

(The `--backend gateway` flag is required: the CLI defaults to the deterministic offline mock, so omitting it silently reproduces the mock arm instead of the published real-gateway numbers.)

F.3 Section 5.7.1 real-gateway compiler/hot-path harness

```
./venv/bin/python examples/paper_real_llm_compiler_test.py \
    --backend real --scale paper
```

(the `mock` backend reproduces the offline variant; `--scale paper` selects the 50-realization, 40-instance protocol of the paper run).

F.4 Section 5.8 four-task demonstration

```
./venv/bin/python examples/system_task_demo.py \
    --seed 42 --files 6 --lines-per-file 400 --repeats 12 \
    --hot-threshold 5 --output examples/system_task_results.json
```

with test suite `pytest tests/system/test_system_task_demo.py`.

F.5 Section 5.10–5.11 Extension-Experiment Commands (Offline)

The extension experiments behind Sections 5.10 and 5.11 (Appendices I.1–I.2) are offline and deterministic; each results JSON records its own provenance (seeds, scale, `git_rev`: `0e1671f8` for the original runs, `c55dc1db` and `b6af966d` for the two review-motivated controls of I.4). Seed and scale conventions, mirrored from `REPRODUCING.md`: the trace census analyzes the corpus regenerated by `examples/random_tasks_stress.py` (master seed 2026, per-task corpus seeds 10000–10099); the capability calibration uses pool seed base 2000, enrichment base 5000, cache-workload base 1000, 5 seeds per condition; the selector bench draws `Random(size*1000+seed)` per cell over sizes 20/50/100/200 × seeds 0–4; the EMA stress uses RNG seeds 1000–1004 (threshold 100, stability 0.9, EMA window 10, γ 0.181818, share threshold 0.3); the posture matrix uses task seeds 11–15; the namespacing experiment seeds 1000–1004 (30 cohorts × 100 items, 13,000 lookups); the replay bench draws unseen values with `Random('e7-replay-generalization-seed{seed}')`, experiment seeds 0–4; the fusion-chain bench uses chain-arm seed base 5000 and control base 2000 over lengths 50/200/1000 × 4 levels × 5 seeds; the token-mix sweep uses seeds 1000–1004, 13 mix points × 3 skews × 200 draws (39,000 draws). Command lines:

```
./venv/bin/python examples/analyze_task_traces.py \
    --trace-dir examples/random_task_traces --top 10 --output <path>

./venv/bin/python examples/benchmarks/capability_calibration.py

./venv/bin/python examples/benchmarks/selector_dp_bench.py

./venv/bin/python examples/benchmarks/hotpath_ema_stress.py

./venv/bin/python examples/benchmarks/security_posture_matrix.py \
    --seeds 5 --tasks-per-scenario 40

./venv/bin/python examples/benchmarks/cache_namespace_experiment.py

./venv/bin/python examples/benchmarks/replay_generalization_bench.py

./venv/bin/python examples/benchmarks/fusion_chain_bench.py

./venv/bin/python examples/benchmarks/promptcache_sweep.py
```

The raw-results file backing each experiment is listed in Appendix I (I.1–I.2) and in `REPRODUCING.md`.

F.6 Appendix I.3 Live-Gateway Re-Run Commands

Two live runs back Appendix I.3; both target the gateway at `https://127.0.0.1:8443` (model `claude-glm-5.3-flash`, temperature 0), and live model replies are their only non-deterministic input—the aggregates and analysis recompute offline from the recorded raw replies (`REPRODUCING.md` records the scale decisions and the gateway-quota episode; Appendix I.3 the protocol). The cache bench is invoked once per Zipf level—repeat the command below with `--zipf 1.0` / `--zipf 1.5` and a matching `--output`, one raw chunk file per invocation (seeds {0, 1} with `rng = Random(1000 + seed)`); `--backend gateway` is required because the CLI

defaults to the offline mock—and the postprocess command folds the three raw chunk files into `gateway_multiseed_results.json`:

```
./venv/bin/python examples/paper_real_llm_quality_check.py --backend real \
    --scale tiny --tasks 1 --max-tokens 4096 \
    --json examples/benchmarks/gateway_quality_probe.json
./venv/bin/python examples/paper_real_llm_quality_check.py --backend real \
    --scale tiny --max-tokens 3072 \
    --json examples/benchmarks/gateway_quality_run_raw.json
./venv/bin/python examples/benchmarks/e10_gateway_postprocess.py \
    --quality-probe examples/benchmarks/gateway_quality_probe.json \
    --quality-run examples/benchmarks/gateway_quality_run_raw.json \
    --i9-baseline examples/paper_real_llm_quality_results_real.json \
    --multiseed-run examples/benchmarks/gateway_multiseed_raw_zipf0.5.json \
                    examples/benchmarks/gateway_multiseed_raw_zipf1.0.json \
                    examples/benchmarks/gateway_multiseed_raw_zipf1.5.json \
    --paper-bench examples/benchmarks/isa_real_llm_results.json \
    --quality-out examples/benchmarks/gateway_quality_results.json \
    --multiseed-out examples/benchmarks/gateway_multiseed_results.json

./venv/bin/python examples/benchmarks/isa_real_llm_bench.py \
    --backend gateway --tasks 100 --seeds 2 --zipf 0.5 --max-tokens 64 \
    --output examples/benchmarks/gateway_multiseed_raw_zipf0.5.json
```

F.7 Artifact Contents Inventory

Per-file inventory of the artifact (mirrored from `REPRODUCING.md`):

- the AgentISA implementation (`agentflow.core.isa`);
- the microbenchmark suite (`examples/benchmarks/isa_microbench.py`);
- the real-LLM benchmark (`examples/benchmarks/isa_real_llm_bench.py`);
- the system-level task demonstration (`examples/system_task_demo.py`, raw results in `examples/system_task_results.json`);
- the four-task workload characterization harness (`examples/system_task_workload.py`, tests in `tests/system/test_system_task_workload.py`);
- the real-gateway compiler/hot-path harness (`examples/paper_real_llm_compiler_test.py`), its offline-mock and real-gateway results (`examples/paper_real_llm_compiler_results_mock.json`, `examples/paper_real_llm_compiler_results_real.json`), and tests (`tests/system/test_real_llm_compiler_test.py`);
- the raw results (`examples/benchmarks/isa_microbench_results.json` and `examples/benchmarks/isa_real_llm_results.json`);
- the 43 multi-agent functional tests (`tests/core/isa/test_coordination.py`, `tests/core/instructions/test_agent_coordination.py`).

F.8 Artifact Notes: Superseded Implementations

All numbers in the main text's Section 5 were produced after the soundness fixes below; no superseded implementation contributes to any reported figure.

Folding pass (Section 5.3). An earlier implementation folded more aggressively—44.4% at length

200—but could place side-effectful instructions into shared groups when types were compatible, violating the disjointness condition; it was replaced by the current strictly disjoint design, and all reported figures use the current pass.

Domain-score weighting (Section 3.5). An earlier form folded the domain weight w_k into s_k as well—double-counting it once at aggregation and again at the top level; Section 5.6's injected-score tests were unaffected (they bypass aggregation).

Pattern-recognizer prototype (Section 3.3.1). An earlier prototype (TF-IDF embeddings, density-based clustering, k-means) was abandoned—non-deterministic across library versions, degenerate on short traces (a k-means view remains in `pattern_mining`, fixed `random_state`).

F.9 Mechanism Status Table

The paragraph below is reproduced verbatim from the main text's Section 7, where it now appears in summary form:

Implementation status of the contributions. A per-mechanism status table is in supplementary F.9. For auditability, the five contributions above split by status: *implemented and exercised by the benchmarks*—the instruction set and contracts, Agentic Calling, result caching, fusion/DCE/grouping, hot-path detection and adoption (exact default), the coordination layer with injected-score capability selection, and the security monitor (fail-open default; 43 tests); *implemented with known limits, exercised by the Section 5.11 mechanism experiments but not the Section 5.2–5.5 benchmark arms*—the opt-in DP tree-pattern selector (default heuristic), the EMA hot-path detector (LRU-bounded tracked-path state; default exact), the strict cache namespace (default params-only key), the strict security mode, parameter-binding composite replay (mined via the learner's threshold-validated path, not the approval gate; constructor-baked parameters not rebindable), and capability metric extraction (static, declared-metadata only; coordinator wiring still injected); *implemented with known limits, still unexercised by any Section 5 experiment*—the bounded checker (pure instructions, bounded state spaces only) and runtime context validation (deletion blind spot disclosed); *specified, not implemented*—cycle-accurate pipeline scheduling, block-granular trace formation, adaptive performance-triggered recompilation, and symbolic-equivalence checking. Theorems 3.4 and 3.6 are conditional arguments for the implemented rewrites; Theorem 3.5 covers the shipped unit-slot dispatch, and the specified components carry no verification claim.

Per-mechanism companion to the status statement of the main paper's Section 7: each mechanism, its implementation status, and the experiment that exercises it. The opt-in rows are exercised by the Section 5.11 mechanism experiments, not by the Section 5.2–5.5 benchmark arms.

Mechanism	Status	Exercised by
result caching	implemented	§5.2 + 5.7
static fusion + DCE	implemented	§5.3 + 5.7.1
hot-path specialization	implemented	§5.4
coordination layer	implemented, injected scores	§5.6
security monitor	implemented, fail-open default	§5.6
DP tree-pattern selector	implemented, opt-in	§5.11
EMA hot-path detector	implemented, opt-in	§5.11
strict cache namespace	implemented, opt-in	§5.11

G Priced-Comparison Protocol and Table

Full protocol of the priced comparison summarized by the main paper's Section 5.7.1; all costs are MODELED from a pricing overlay on measured tokens.

A priced comparison (costs MODELED—computed from a pricing overlay rather than observed—on measured tokens). The harness prices the run's measured token counts under three modes: *no optimization* (every realization re-queries at full price), *AgentISA result caching* (first occurrences only—the measured behavior of both arms), and a *provider prompt cache* (MODELED: every realization queries, but repeat prompts bill input tokens as prompt-cache reads at 0.1x the input price—configurable—mirroring Anthropic's published prompt-caching read discount; completions always bill at full price). Two simplifications—omitting Anthropic's 1.25x cache-write premium and 1024-token minimum cacheable prefix—both favor the provider-cache baseline, so the comparison is conservative for us. List prices are parameterized stand-ins ($3/M input, $15/M output by default) because the gateway's actual pricing is unknown. Token counts are measurements, the pricing overlay is a model, and no dollar-savings claim is made:

Table 2: Measured calls and tokens with a MODELED pricing overlay (real-gateway run, s = 1.0, 50 realizations; the price column is a model, not a quote).

Pricing mode	Calls	Input tokens (full / cache-read)	Output tokens	Modeled cost
No optimization	50	9,018 / 0	25,600	$0.4111
AgentISA result caching	22	4,027 / 0	11,264	$0.1810
Provider prompt cache (MODELED)	50	4,027 / 4,991	25,600	$0.3976

On this run's token mix the result cache cuts modeled cost by 56.0%—it removes whole calls, completions included—while the modeled provider prompt cache cuts only 3.3%, because the reasoning model's completion tokens dominate and a discount on input reads cannot touch them; the ranking of the two caching strategies is a function of the input/output token mix, not a constant. On the deterministic mock at paper scale (200 tasks × 1 seed × 3 Zipf levels, template-accounted tokens, same modes and prices) the result cache cuts modeled cost by 80.1%, 81.6%, and 85.1% at s = 0.5, 1.0, 1.5, while the modeled provider prompt cache cuts 28.9–31.4%—there the short completions do not dominate, so the input-side discount matters more; both columns remain MODELED-on-measured in both cases. GPTCache, an external cache layer, is explicitly out of scope; no head-to-head against it is claimed.

Why the provider-cache column stays MODELED. A direct probe (2026-10-22; artifact `gateway_promptcache_probe.json`) sent an identical ~800-token system+user prefix twice per alias with `cache_control: {type: ephemeral}` blocks on `claude-doubao-seed-2.0-code` and `claude-deepseek-v4-flash`: every response reports plain `input_tokens` (842 and 891, respectively) with `cache_creation_input_tokens = cache_read_input_tokens = 0` on both calls—the gateway does not surface provider prompt-cache billing fields, so the read-discount behavior cannot be measured through it; the MODELED overlay is a necessity of the environment, not a choice. The result-cache column's calls and tokens remain gateway-measured.

H Four-Task Characterization — Full Detail

Task inventory, per-task instruction streams, configuration, single-task walk-through, and both result tables of the main paper's Section 5.8; numbers identical to the main paper.

The four tasks:

- the original **service-log triage** (linear, 7 instructions), detailed below;
- **csv-etl-anomaly** (fan-out, 8): CSV shard ingest, schema guard, parse, per-column statistics, row signatures and anomaly triage, aggregate, write;
- **report-merge** (wide fan-in, 7): five-report ingest, header guard, dedupe, per-line signatures, cacheable consistency check, merge, write;
- **regression-diff** (two-branch convergence, 8): parallel baseline/candidate ingests, guard, normalize, diff, classify hunks, aggregate, write.

The triage task's instruction stream (seven instructions over sandbox-resident files): `ingest_logs` reads six 400-line sandbox log files (side-effectful); `check_syntax` is a defensive format guard that never rejects here (a *planted dead guard*, standing in for real pipelines' belt-and-braces checks); `normalize_entries` parses and normalizes lines; `extract_incidents` lifts them into incident records; `classify_signature` triage-classifies each incident (LLM-type, cacheable, backed by a deterministic call-counting rule stand-in); `aggregate_triage` summarizes by severity; `write_report` writes the report (side-effectful).

Configuration. Seed 42; 6 files × 400 lines; 12 repeated realizations of the task; hot-path threshold 5. Every phase is deterministic and re-runnable from the repository (Artifact Availability).

What the triage walk-through shows.

- **Static compilation with structural validation.** NONE compiles to 7 effective instructions / 7 dispatches; AGGRESSIVE to 6 / 6 (14.3%): the dead guard fuses with the following pure step rather than being dropped. The walk-through *asserts the compiled stream's dataflow validity*—every step whose output is consumed downstream must survive compilation, the only eliminable step being the guard—so the shrinkage cannot come from dropping live producers.
- **Reuse across repeated realizations.** The first realization makes 50 (simulated) model calls; all eleven subsequent realizations make zero (result cache hit rate 91.7% on the classification step); all 12 reports are byte-identical.
- **Closed-loop hot-path adoption.** The stream crosses the hot-path threshold at the sixth recording (the counter must strictly exceed it and recording follows execution, so adoption lands at realization threshold+2); the runtime *adopts* the optimized form automatically, repeats 7–12 execute the shrunken 6-instruction stream (instruction executions per realization: `[7,7,7,7,7,7,6,6,6,6,6,6]`), and on a fresh workspace the optimized form reproduces the report byte-for-byte—semantics-preserving on the real task, not only on synthetic sequences.
- **Full-stack determinism.** A completely fresh stack—new instructions, new cache, new workspace, same seed—reproduces the report byte-for-byte (SHA-256 equality).

The four-task characterization. All four tasks run the identical closed-loop battery (`examples/system_task_workload.py`, `characterize()`, 12 realizations each, hot-path threshold 5, one planted dead guard and one cacheable LLM-type step each, the latter backed by a deterministic rule stand-in counting simulated model calls):

Structure (compile-time):

Table 3: Compile-time structure of the four tasks (12 realizations each; shrink is the NONE → AGGRESSIVE effective-instruction reduction).

Task (shape)	Instrs	Effective count NONE → AGGRESSIVE	Shrink	Dead guard
log triage (linear, 7)	7	7 → 6	-14.3%	`check_syntax`
csv-etl-anomaly (fan-out, 8)	8	8 → 6	-25.0%	`check_schema`
report-merge (wide fan-in, 7)	7	7 → 6	-14.3%	`check_headers`
regression-diff (two-branch, 8)	8	8 → 7	-12.5%	`check_formats`

Execution (runtime, 12 realizations each):

Table 4: Runtime behavior of the four tasks (12 realizations each; classification-type steps are backed by deterministic rule stand-ins).

Task	Calls 1st / re-runs	Cache hit	Hot-path adopted	Reports byte-identical
log triage	50 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
csv-etl-anomaly	14 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
report-merge	16 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)
regression-diff	240 / 0	91.7%	repeat 7	yes (12/12 + fresh-stack SHA)

I Extension-Experiment Protocols and Detailed Tables

Full protocols and detailed tables for the extension experiments whose headlines appear in the main paper's Sections 5.10 and 5.11. Every number below is extracted programmatically from the corresponding committed results file under `examples/benchmarks/` (each file carries a `provenance` block with the git revision `0e1671f8`, UTC timestamps, seeds, and scale; aggregates are computed by the generating script from raw per-instance records, never hand-entered). As elsewhere, N/A results, not-covered axes, and residual failure modes are reported as such rather than dropped.

I.1 Real-Trace Workload Characterization: Protocol and Complete Tables

`examples/benchmarks/trace_workload_results.json` records the census summarized by the main paper's Section 5.10:

- Driver: `examples/benchmarks/trace_workload_characterization.py`
- Analyzer: `examples/analyze_task_traces.py`

Protocol. The corpus is 100 task traces from the seeded random-task stress battery (`examples/random_tasks_stress.py`; generator master seed 2026, per-task corpus seeds

10000–10099, extracted from the generator source at run time—not hand-copied). The analyzer is pure-stdlib and takes the first `kind=workflow` record of each trace file (message records skipped; malformed JSON rejected loudly). The analysis is a deterministic full census of the committed corpus: the analyzer has no sampling randomness, so multi-seed averaging does not apply (the corpus-generation seeds are recorded; `analysis_seed` is null in the provenance for exactly this reason). The census counts 100 files / 100 workflows / 792 steps / 12 distinct instruction signatures and 692 adjacent step pairs; the corpus is pinned by `corpus_sha256` 7c05d307c64ab418f85811efa18489fa80463ab504fdc1edb3f0c22a4bf323eb, and only the timestamp and git revision vary between runs. Instruction signatures are `(type, params)` pairs; because the traces serialize no instruction params, operation identity is recovered from step names by stripping trailing positional index suffixes (`guard_0_2` → `guard`), with an explicit non-empty `params` field canonicalized with sorted keys when present. The Zipf fit is an OLS regression of log-rank on log-frequency over the 12 signature-class frequencies (fewer than two classes → `s`/`R²` null; a zero-variance distribution → `R²` = 0.0 by convention). Two *distinct* reuse quantities are reported: the corpus-level share of step instances that repeat an already-seen signature, and the mean of per-workflow repeat shares. The shape census resolves dependencies (by output name, then step id) into a DAG and classifies workflows by node structure: *linear* (no fan-out/fan-in node), *fan-out* (a node with ≥ 2 consumers), *fan-in* (a node with ≥ 2 dependencies), *branch* (both, e.g. a diamond). Dead-end steps are a single-pass structural count (pure steps whose outputs no step consumes)—not the compiler's AGGRESSIVE fixpoint liveness analysis. The fusibility census mirrors `agentflow/core/isa/compiler.py::_are_fusible` on the trace dicts: same type, both members explicitly pure, and no producer–consumer edge inside the pair.

Table 5: Signature frequency distribution (top 10 of 12 classes; the two remaining classes account for the residual 74 instances, 792 − 718)

Rank	Signature (type:operation)	Instances
1	CONTROL_FLOW:guard	152
2	STATE_MANAGEMENT:ingest	100
3	STATE_MANAGEMENT:write_report	100
4	LLM_INTERACTION:classify	68
5	CONTROL_FLOW:firsthalf	61
6	CONTROL_FLOW:reverse_words	56
7	CONTROL_FLOW:filter_warn	53
8	CONTROL_FLOW:tail	43
9	CONTROL_FLOW:filter_info	43
10	CONTROL_FLOW:dedupe	42
11–12	two further classes (combined)	74

Table 6: Census summary

Quantity	Value
Trace files / workflows	100 / 100
Steps (instances)	792
Distinct (type, params) signatures	12
Adjacent step pairs	692
Zipf fit (log-rank/log-freq OLS)	$s = 0.593$ (0.5927), $R^2 = 0.971$ (0.9714)
Corpus-level repeated instances	780 / 792 (98.5%)

Quantity	Value
Per-workflow mean repeat share	18.3%
Shape census (linear / fan-out / fan-in / branch)	21 / 79 / 0 / 0
Average steps per workflow	7.9
Dead-end steps (single-pass count, pure, no consumer)	152
Workflows with multiple components (disconnected)	0
Fusible adjacent pairs	106 / 692 (15.3%), all CONTROL_FLOW

Disclosures. The corpus is generator-produced, not production logs; the census characterizes workload structure only. The dead-end count is deliberately the weaker single-pass structural count, so its exact agreement with the 152 planted guards is a cross-validation, not a compiler-behavior claim. The corpus contains no fan-in or branch workflows, so those shape rows report the census honestly at zero rather than by construction. Repetition is dominated by cross-task vocabulary (98.5% corpus-level) while intra-task duplication stays low (18.3% per workflow)—the two definitions answer different questions and both are reported.

I.2 Mechanism-Extension Experiments: Complete Protocols and Detailed Tables

The six experiments whose headlines the main paper's Section 5.11 sums up, one per shipped opt-in mechanism, plus the two extensions of existing benchmark protocols cited in the same section (chain-enriched workloads, extending the Section 5.3 optimization benchmark; the token-mix sweep, extending the Section 5.7.1/Appendix G priced comparison). Each experiment's results file records the dependency component as shipped in its `coverage` block; N/A results and not-covered axes are listed, not dropped.

I.2.1 Capability-Score Calibration

Raw results: `examples/benchmarks/capability_calibration_results.json` (script `capability_calibration.py`). **Question:** do the statically extracted capability scores of Section 3.5 track measurable mock-backend behavior? **Pool:** the Section 5.3 microbench generator (`synthetic_formal_sequence`, length 200, `p_dead` 0.25, `p_pure` 0.55, pool seed base 2000 mirroring the compiler experiment), 5 seeds per condition; enrichment draws use an independent seed base 5000 (paired design: the same seed sees identical pools and workloads across intensities), and the cache-workload seed base 1000 mirrors the Section 5.2 cache experiment. **Protocols:** 8 arms, 165 replicas in total—un-enriched base (5), and graded declaration-injection arms P_flow / P_cache / P_resource / P_security / P_audit (25 each), P_mix (20), and a generation anchor G_anchor (15)—over intensities {0.0, 0.25, 0.5, 0.75, 1.0}, four dead/pure mixes, generation anchor lengths {50, 200, 1000}, a cache workload of 2,000 tasks × 20 parameter values × Zipf {0.5, 1.0, 1.5}, and a Zipf-1.0 generation workload; plus 10 inertness probes. **Instruments (all real code paths):** the mock backend (one simulated round trip per executed LLM-typed instruction, 80 simulated ms counted, never slept), the cache arm (honoring declared `optimization.cacheable`), the compiler battery (`compile()` at BASIC/ADVANCED plus isolated fusion-only and DCE-only passes and the resource plan's total cost), and the executor gate battery (`FormalInstruction.execute` against an unprivileged, resource-starved context). Extraction is static and side-effect-free; every pool, enrichment, and workload is seeded. Wall time 2.72 s; git rev 0e1671f8.

Extraction is no longer constant-zero. On un-enriched base pools, 7 of 17 metrics and 5 of 8 domain scores are already nonzero (per-seed nonzero counts 7/17 and 5/8 in all five seeds; the stub era was 0/17 by construction), and all 17 metrics attain a nonzero value somewhere across the protocol and probe arms.

Table 7: Domain-score correlations (Pearson r / Spearman ρ; all cross-path coefficients significant at $p < 10^{-12}$, generation at $p = 4.6 \times 10^{-4}$; the declared-and-enforced couplings are exact)

Domain	Measured counterpart	Coupling	r	ρ	n
reasoning	fusion-pass instruction-count reduction	cross-path	−0.9647	−0.9526	25
memory	mock-backend cache hit rate (Zipf 1.0)	cross-path	+0.9482	+0.9517	25
generation	uniform model-call share (one execution pass)	same quantity via real execution	+0.7902	+0.9034	15
coordination	parallel dispatch compression (ADVANCED)	cross-path	−0.9767	−0.9836	25
optimization	executor resource-gate rejection rate	declared-and-enforced	−1.0000	−1.0000	25
security	executor security-gate rejection rate	declared-and-enforced	+1.0000	+1.0000	25
specialization	— (N/A: no task-level ground truth in the microbench)	—	—	—	—
robustness	— (N/A: no runtime enforcement axis)	—	—	—	—

Secondary arms, reported for honesty: memory vs cache hit rate at Zipf 0.5/1.5 gives $r = +0.9843$ / $+0.8925$; optimization vs the compiler resource-plan's total cost gives -1.0000 and vs cache hit rate $+0.9490$; generation under a Zipf-1.0 *workload* arm (rather than the uniform pass) drops to $r = +0.176$ (workload-position concentration adds variance beyond pool composition—a weaker coupling is expected there).

Table 8: Per-metric correlations (eight correlated; nine N/A with probe evidence)

Metric	Arm	Measured quantity	r	n
causal_understanding	P_flow	fusion-only reduction	−0.9647	25
long_term_memory	P_cache	cache hit rate (Zipf 1.0)	+0.9482	25
creativity	G_anchor	uniform model-call share	+1.0000	15
coherence	P_mix	DCE-only reduction	−0.9900	20
efficiency	P_cache	cache hit rate (Zipf 1.0)	+0.9490	25
resource_usage	P_resource	resource-gate rejection rate	−1.0000	25
collaboration	P_flow	parallel dispatch compression	−0.9767	25
policy_compliance	P_security	security-gate rejection rate	+1.0000	25
logical_consistency	N/A	structural well-formedness has no runtime reader (probe: `malformed_dependencies` moves the score, no measured output changes)	—	—
abstract_reasoning	N/A	declared parameter schemas consulted by no microbench path (probe `parameter_schemas`: zero behavioral delta)	—	—
working_memory	N/A	declared input arity consumed by no runtime path; the extractor-counted `input:` prefix is not matched by the compiler's bare-name dataflow rules (probe `prefixed_input_deps`)	—	—
domain_expertise	N/A	category tags inert; no task-level ground truth (probe `category_tags`)	—	—

Metric	Arm	Measured quantity	r	n
adaptability	N/A	deadline/locality hints read by no runtime path (probe `adaptability_hints`)	—	—
communication	N/A	no inter-agent protocol in the microbench (probe `agent_type_recategorization`)	—	—
error_handling	N/A	declared timeouts/retries not enforced by the formal executor (probe `error_contract`: contract broken on every instruction, outcomes unchanged)	—	—
recovery	N/A	recovery metadata extracted, no recovery machinery to measure (probe `recovery_metadata_key`)	—	—
threat_detection	N/A	no measured counterpart (probes `security_verification_flag` + protocol P_audit; see below)	—	—

The creativity row is instrument agreement—the same quantity (declared LLM composition share) measured through the real mock-backend execution path—and is labeled as such rather than as behavioral prediction. **Inertness probes:** 9 of 10 injected declaration families produce a measured delta of exactly 0.0 in the microbench batteries (`parallelizable_hint`, `security_verification_flag`, `category_tags`, `recovery_metadata_key`, `parameter_schemas`, `malformed_dependencies`, `error_contract`, `adaptability_hints`, `prefixed_input_deps`); the one with a detected effect is `agent_type_recategorization` (maximum delta 0.035 on `basic_reduction` / `advanced_effective_reduction` / `fusion_only_reduction`), i.e., fusion keys on instruction *type* as an adjacency rule—a compiler artifact, not inter-agent coordination semantics, which is why communication stays N/A rather than inheriting that coefficient. **threat_detection reclassified N/A, with measured evidence:** across graded audit intensities the extracted score spans [0.000, 1.000] while every compiler measurement stays invariant per seed—the compiler's audit branch is unreachable-and-redundant (`_has_side_effects` returns True both through the audit branch and the final fallback, and microbench pools declare metadata `side_effects` regardless), and the executor's `_verify_`-

`security` does not read `security.verification`; microbench pools also contain no SECURITY-typed instructions. Specialization and robustness are reported N/A with probe evidence rather than fabricated coefficients.

I.2.2 DP Tree-Pattern Selection Benchmark

Raw results: `examples/benchmarks/selector_dp_results.json` (script `selector_dp_bench.py`). **Question:** does the opt-in MATCH-TREE selector (Appendix D.1) cover random IR trees at cost at or below the default heuristic? **Instances:** 20 = sizes {20, 50, 100, 200} nodes × seeds 0–4, trees grown per-node for exact node counts (fanout ≤ 3, depth capped; observed depth mean 8.75, max 11), rng seed = `size × 1000 + seed` per (size, seed) with catalogue and tree drawn from independent streams of the same seed. **Catalogue per instance:** one base single-node instruction per tree op (cost U(0.5, 5.0)); 2 premium variants (1.3–2.0 × base cost, carrying an `urgency` parameter that the score-first heuristic values but the cost-minimizing DP does not); 3–6 fused multi-node patterns declaring `params["spanned_child_ops"]`, priced at 0.70–0.95 × the base cost of (root + spanned child ops) while covering the spanned children's whole subtrees. **Arms:** `InstructionSelector(selection_strategy="dp")` vs the default heuristic, both through the production `select_for_tree` API. Timing: 5 reps per arm, median reported. Coverage complete 20/20 (pre-bench component check recorded in provenance: the selector's target suite, 37 passed). Both arms produced complete, valid tilings (each node covered exactly once) on all 20 instances.

Overall: DP wins 20/20 (0 ties, 0 losses); covering cost DP 85.80 ± 73.69 vs heuristic 292.08 ± 244.95 (gap 206.28 ± 203.43); ratio DP/heuristic mean 0.361, range 0.054–0.685. Selection time DP 0.241 ± 0.163 ms vs heuristic 0.265 ± 0.194 ms; time-ratio mean 0.950, range 0.801–1.086—the DP was *not* slower in this environment, recorded as found (the design allows slower). **Mechanism diagnostics:** the DP used 80 fused sites in total and selected premium variants 0 times; the heuristic used 0 fused sites and 251 premium sites.

Table 9: By tree size (5 instances per row; cost ratio is DP/heuristic)

Nodes	DP cost (mean ± sd)	Heuristic cost (mean ± sd)	Cost ratio mean (range)	DP wins	DP time (ms)	Heuristic time (ms)	Time ratio
20	33.85 ± 8.49	61.53 ± 11.59	0.546 (0.484–0.592)	5/5	0.066 ± 0.002	0.066 ± 0.004	1.011
50	47.10 ± 31.95	150.82 ± 22.40	0.305 (0.054–0.555)	5/5	0.141 ± 0.015	0.144 ± 0.003	0.981
100	119.22 ± 84.05	289.71 ± 69.24	0.379 (0.204–0.685)	5/5	0.272 ± 0.017	0.286 ± 0.001	0.951
200	143.01 ± 83.11	666.24 ± 114.78	0.213 (0.080–0.365)	5/5	0.484 ± 0.019	0.563 ± 0.003	0.859

Table 10: Per-instance detail (covering costs in catalogue units)

Size	Seed	DP cost	Heuristic cost	Ratio (DP/Heur)	Gap (Heur − DP)
20	0	41.163	70.245	0.586	29.082
20	1	28.932	53.585	0.540	24.653
20	2	24.850	47.014	0.529	22.164
20	3	29.795	61.614	0.484	31.819
20	4	44.502	75.207	0.592	30.705
50	0	8.280	153.381	0.054	145.101
50	1	21.207	113.463	0.187	92.256
50	2	83.754	150.826	0.555	67.072
50	3	51.958	168.336	0.309	116.378
50	4	70.317	168.089	0.418	97.772
100	0	241.968	353.385	0.685	111.417
100	1	58.532	226.463	0.258	167.931
100	2	45.249	222.026	0.204	176.777
100	3	169.694	369.600	0.459	199.906
100	4	80.655	277.086	0.291	196.431
200	0	50.272	631.037	0.080	580.765
200	1	142.518	848.946	0.168	706.428
200	2	241.602	662.779	0.365	421.177
200	3	208.490	656.266	0.318	447.776
200	4	72.176	532.196	0.136	460.020

Disclosure. The invariant under test is structural—DP cost $\leq$ heuristic cost, since the DP minimizes over a covering set that includes the greedy single-node covering; the *absolute* gap depends on this synthetic catalogue's pricing model (a fused site near the root subsumes a large subtree by construction of the discount), so the gap size must not be read as a deployment speedup. No benchmark in Section 5 runs the DP arm; every published number uses the default heuristic.

I.2.3 EMA Hot-Path Detection Under Trace Diversity

Raw results: `examples/benchmarks/hotpath_ema_results.json` (script `hotpath_ema_stress.py`). **Question:** does the opt-in EMA trigger (`detection_mode="ema"`, Appendix D.4) promote diverse trace variants the exact default misses, without false promotions on one-shot diversity? **Design:** 7 seeded stream scenarios × 5 rng seeds (1000–1004), each stream fed identically to the exact detector, the EMA detector, and a deterministic read-only oracle (a path is hot iff it has ≥ 2 recordings and its cumulative share exceeds θ; brute-forced over the stream). Detector configuration: hot-path threshold 100, stability bound 0.9, EMA window 10 ($\gamma = 0.181818$), share threshold 0.3, default tracked-path cap 1000 in the detection cells. Adoption is measured end-to-end through `OptimizationManager` (detection → trace re-optimization → first shortened-form service). Cap stress: a Zipf(1.1) mix (~800 recordings) interleaved with 2,200 one-off paths (~3,000 recordings, 2,210 distinct paths), caps swept 2000 → 100; cap rescue: an 80-path warmup + burst stream, caps 1000 → 10. Scale: 175,680 detector `record` calls in total (detection cells 6,800; adoption cells 880; cap cells 75,000; rescue cells 5,600), wall 0.33 s; 12 hard invariants are asserted before the results file is written, and two full reruns are byte-identical apart from timing. Block-granular trace formation (D.5) remains specified, not implemented—the adoption path measured is the shipped whole-sequence re-optimization.

Table 11: Scenario results (promoted-path counts, mean ± sd over 5 seeds; recall is vs the oracle; false promotions 0.0 ± 0.0 for both detectors in every scenario)

Scenario (stream)	Exact	EMA	Oracle	EMA recall
concentrated (512 recordings, 18 paths)	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
diverse_rotated (120; 3 variants × 40, far below threshold 100)	0.0 ± 0.0	3.0 ± 0.0	3.0 ± 0.0	1.0 ± 0.0
diverse_zipf (~160; Zipf(1.1) over 10 ranks)	0.0 ± 0.0	1.8 ± 0.45	2.2 ± 0.45	0.867 ± 0.298
cyclic_shift (160; 4 shifting variants)	0.0 ± 0.0	2.0 ± 0.0	2.0 ± 0.0	1.0 ± 0.0
bursty (88; 80-call burst below threshold)	0.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0
warmup_heavy_burst (160; 80 one-shot warmups, default cap)	0.0 ± 0.0	0.0 ± 0.0	1.0 ± 0.0	0.0 ± 0.0
pure_diversity (160 one-off trajectories)	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	—

Adoption latency. On the concentrated stream the EMA detector promotes at recording 2.0 (mean over 5 seeds) where the exact detector first promotes at recording 107.0; on the bursty stream the EMA detects the burst at recording 13.0 and adopts immediately—12.0 full-form executions before adoption, the served trace shrinking 4 → 1—in 5/5 seeds, while the exact detector stays cold in 0/5. **Warmup disclosure (a measured implementation property, reported as found):** the EMA detector decays each path's counter only on that path's own recordings and normalizes by the sum over LRU-tracked paths, so 80 one-shot warmup paths pin the share denominator at 85.5 and the later burst stays below threshold under caps ≥ 15 (detected 0/5 at caps 1000/100/50/20/15, denominators 85.5/85.5/54.5/24.5/19.5); tightening the cap evicts stale paths, shrinks the denominator, and restores detection—cap 12: 5/5 detected at recording 90.0 (denominator 16.5, 69 evictions); cap 10: 5/5 at 87.0 (denominator 14.5, 71 evictions). The exact detector promotes nothing in the rescue stream (0.0 in all cells).

Table 12: Tracked-path cap sweep (2,210 distinct paths recorded; evictions and hot-path losses, mean over 5 seeds; all five per-path mappings respect the cap in 5/5 seeds at every cap)

Cap	Evictions	Exact ever-hot → lost	EMA ever-hot → lost
2000	210	2.0 → 0.0	0.6 → 0.0
1000	1210	2.0 → 2.0	0.6 → 0.6
500	1710	2.0 → 2.0	0.6 → 0.6
200	2010	2.0 → 2.0	0.6 → 0.6
100	2110	2.0 → 2.0	0.6 → 0.6

The cap is a real bound with a real cost: hot paths themselves are evictable, and at cap ≤ 1000 both detectors lose every path they had promoted (the exact detector's 2.0 promoted hot paths, the EMA's 0.6). Eviction is LRU (least-recently-*recorded*), and an evicted path re-accumulates from zero if it recurs.

I.2.4 Strict-Security Posture Matrix

Raw results: `examples/benchmarks/security_posture_results.json` (script `security_posture_matrix.py`). **Question:** what does the opt-in strict (fail-closed) flag of `SecurityMonitor`/`AgentCoordinator` change relative to the fail-open default? **Design:** 5 scenario rows (4 main—unregistered resource, unknown data sensitivity, incomplete resource-access policy, normal task—plus 1 supplementary: a type-malformed `allowed_resources` registry), 18 sub-variants in total; 5 seeds (11–15) $\times$ 40 tasks per scenario = 1,000 task dicts; every task dict is evaluated in *both* modes (paired 2 $\times$ 2 contingency, fresh `SecurityMonitor` and fresh `AgentCoordinator` per task $\times$ mode): 2,000 monitor evaluations and 2,000 end-to-end coordinator runs. Mode flags: `strict=False` (fail-open default) vs `strict=True` (`security_strict` on the coordinator). Every task holds the non-varied security dimensions at known-good values, so exactly one dimension varies per scenario row; the decision function is deterministic, and per-seed allow-rate stdev is 0.0 (seeds vary the task surface—names, request shapes, variant mix—not the verdict). The study is qualitative-and-counting only: no latency, throughput, or memory quantity is measured or claimed, by construction.

Table 13: Posture matrix (200 tasks per row; reason class is the recorded `violation_history` category)

Scenario	Mode	Allowed / denied	Deny rate	Reason class
unregistered_resource	fail-open	0 / 200	1.000	generic_invalid_resource_access
unregistered_resource	fail-closed	0 / 200	1.000	unregistered_resources_strict
unknown_sensitivity	fail-open	200 / 0	0.000	(permitted)
unknown_sensitivity	fail-closed	0 / 200	1.000	unknown_sensitivity_strict
incomplete_policy	fail-open	200 / 0	0.000	(permitted)
incomplete_policy	fail-closed	0 / 200	1.000	incomplete_policy_strict
normal_task	fail-open	200 / 0	0.000	(permitted)
normal_task	fail-closed	200 / 0	0.000	(permitted)
malformed_registry_supplementary	fail-open	0 / 200	1.000	generic_invalid_resource_access
malformed_registry_supplementary	fail-closed	0 / 200	1.000	incomplete_policy_strict

Mode divergence (per-task paired 2 $\times$ 2): unknown_sensitivity and incomplete_policy each produce 200 task pairs that fail-open permits and fail-closed denies—400 of 1,000 paired tasks in total, the actual fail-open holes; unregistered_resource and the malformed-registry row are denied in both modes (200/200 each), but only fail-closed gives a distinguishable reason class; normal

tasks are permitted in both modes (200/200, zero false denials on the well-specified population). **Auditability:** all 800 strict-mode denials in the non-normal scenarios are recorded in `violation_history` with a "(strict mode)" marker (rate 1.000); denials are recorded in *both* modes—what fail-closed adds is the distinguishing reason classes and the marker, not recording itself. **Propagation:** the `AgentCoordinator` gate agrees with the monitor verdict on all 2,000 paired runs (execute-rate equals allow-rate per scenario × mode in all 10 cells). **Not covered:** the always-on authentication/authorization checks are held fixed (all tasks carry tokens; not a matrix axis); `required_level` is LOW everywhere except two normal-task variants (MEDIUM/HIGH), so the sensitivity-vs-requirement mismatch path is exercised only by the shipped unit suite, not scaled here; the supplementary malformed-registry row is grouped by the strict mode under "incomplete policy", while its fail-open outcome is a generic deny (a string registry degenerates to a character set)—unlike the canonical missing-registry hole, which fail-open permits.

I.2.5 Cache-Key Namespacing Under Semantic Drift

Raw results: `examples/benchmarks/cache_namespace_results.json` (script `cache_namespace_experiment.py`). **Question:** under a deployment-side change of semantic configuration (the Section 5.2 cache-key contract's template version / decoding axes), do strictly namespaced keys invalidate while legacy params-only keys keep serving stale entries? **Design:** 3 drift axes—`template_version` v1 → v2, `decoding` temperature 0.0 → 0.7 (both semantic), and a benign control (decoding-dict key insertion order only)—× 2 arms (`strict_namespaced`: one shared `CacheKeyNamespace` object per cohort, mutated in place at the drift event, exactly a deployment's bookkeeping; `regular_legacy`: the historical params-only keys)—× 5 seeds (1000–1004) × 100 real `CompositeInstruction` items per cohort = 30 cohorts, 3,000 instruction instances, 13,000 cache lookups (cold, warm, two post-drift, plus a rollback lookup for strict cohorts on the semantic axes), all through the production `Instruction._get_cache_key/build_cache_key/InstructionCache` paths; cache `max_size` 1000, `ttl` 3600. **Measurement protocol:** a lookup is a miss iff the wrapped leaf body actually re-executed; a served result is stale iff its embedded semantic-config snapshot differs canonically from the deployment's current config—no clock arithmetic, no inferred staleness.

Table 14: Arm × axis results (mean ± sd over 5 seeds; every cell's stdev is 0.0000)

Arm	Axis	Post-drift hit rate	Invalidation rate	Stale-serve rate	Fresh-serve rate
strict namespaced	template_version v1 → v2	0.0000	1.0000	0.0000	1.0000
strict namespaced	decoding 0.0 → 0.7	0.0000	1.0000	0.0000	1.0000
strict namespaced	benign key reorder (control)	1.0000	0.0000	—	—
regular legacy	template_version v1 → v2	1.0000	0.0000	1.0000	0.0000
regular legacy	decoding 0.0 → 0.7	1.0000	0.0000	1.0000	0.0000

The benign-reorder control also reports a key-change rate of 0.0000 (the normalized decoding hash folds dictionary order away, so reordering neither invalidates nor even re-keys). **Rollback**

(strict arm): switching back to the original configuration hits the resident original entries in 100% of lookups on both semantic axes (original-serve rate 1.0000)—namespacing is *isolation, not eviction*: entries from different semantic configurations coexist, and drift is reversible. The legacy arm's post-drift hits are the expected legacy behavior being quantified, not a bug in that arm: every post-drift hit serves an entry produced under the pre-drift configuration. **Not covered:** `model_alias` drift (the third namespace component); shared-store cross-instruction key collisions (covered by the unit suite `tests/core/isa/test_cache_namespacing.py`); TTL/LRU dynamics (the configuration far exceeds the two keys each per-item cache ever holds—drift dynamics are the subject here). Provenance records module digests for `instruction_cache.py` (99718b9af540bba1ab5df8118c5288271545cdde163dd58cfc13c3da8f8114a3) and `instruction.py` (b3657344469bb89937512d3f0bdd37c7d9e00bc5331d2cd8bc79d56ae09f12a5).

I.2.6 Parameterized Replay on Unseen Values

Raw results: `examples/benchmarks/replay_generalization_results.json` (script `replay_generalization_bench.py`). **Question:** does a learned composite instruction, mined from traces that recorded only a small grid of parameter values, replay exactly like re-running its recorded constituent sequence for parameter values never observed during mining? **Streams:** 3 deterministic multi-operator compositions, rotated by seed (`composition_for_seed(seed) = ('filter3', 'scale4', 'chain5')[seed % 3]`): `filter3` (parameters seed, cutoff; mining grid seed {1, 2, 3} × cutoff {10, 20}), `scale4` (+ factor {2, 3}), and `chain5` (+ shift {1, 2}; 5 steps, 4 external parameters). **Unseen draws:** per experiment seed, 210 unseen parameter combinations (minimum required 200) from ranges disjoint per coordinate from the recorded grid—seed 100–499, factor 4–13, shift 3–15, cutoff 21–98—with four guards enforced per case: per-coordinate disjointness, tuple uniqueness, never a recorded tuple, and the per-seed minimum; the draw rng is `random.Random('e7-replay-generalization-seed{seed}')` with recorded draw order. **Conventions:** each case executes the learned instruction under a rotating calling convention—`wrapped` (`{'ctx': {...}}`, the `run_stream` convention the mining traces were recorded under), `flat` (the bare parameter dict), and `sibling` (`{'ctx': {}, <params>}`, the schema-parameter binding path added by the replay-parameterization work). **Oracle:** per case, the recorded constituent sequence is re-run from scratch via `record_trace` with the unseen values as the initial shared context and the step outputs merged; the replay result must equal that merged dict exactly (no tolerance).

Result: 1,050 of 1,050 unseen cases match the oracle exactly (100.0%; 0 mismatches, 0 error cases); per-seed agreement 1.0000 ± 0.0000; per convention 350/350 for each of wrapped, flat, and sibling; per composition filter3 420/420 (seeds 0, 3), scale4 420/420 (seeds 1, 4), chain5 210/210 (seed 2); 179 empty-result edge cases (`kept == []`) all agree. Two independent full runs are byte-identical apart from the timestamp, git revision, and command line.

Table 15: Per-seed detail

Seed	Composition	Cases	Match	Empty-kept cases	Conventions (w/f/s, all matching)
0	filter3	210	210	36	70 / 70 / 70
1	scale4	210	210	31	70 / 70 / 70
2	chain5	210	210	38	70 / 70 / 70
3	filter3	210	210	41	70 / 70 / 70
4	scale4	210	210	33	70 / 70 / 70

Not covered: dialogue-mined templates with an attached `executor` (the replay arm's sibling feature)—no recorded composite body exists to re-run, so there is nothing to compare; constructor-bound parameters (values baked into instruction state at construction time cannot be rebound—replay re-runs the recorded operator *instances*, so only context-carried values can vary; the streams here read every parameter from the shared context, which is the dataflow contract that makes parameterization possible at all); real-workload and LLM streams (the measured streams are deterministic synthetic arithmetic, and the claim measured is exact per-case equality under that contract, nothing broader).

Hot-path trace shrinkage (moved verbatim from the main text's Section 5.4; the main text retains the prose interpretation).

Table 16: Hot-path trace shrinkage per seed (length-100 traces, detection threshold 50 recordings). The replay column records the executed-state identity check: the optimized trace, re-executed against an independent state, reproduces the original trace's final state exactly in every seed.

Seed	Original length	Optimized length	Shrink ratio	Replay state identical
0	100	30	70.0%	yes
1	100	41	59.0%	yes
2	100	30	70.0%	yes
3	100	36	64.0%	yes
4	100	34	66.0%	yes

Optimization-level sweep (moved verbatim from the main text's Section 5.3; the main text retains the prose interpretation).

Table 17: Effective-instruction-count reduction by optimization level and sequence length (5 seeds, mean ± sample standard deviation; planted dead 0.25 / pure 0.55 workload).

Sequence length	Level	Effective-instruction reduction	Dispatch count
50	NONE	0% ± 0.0	50.0 ± 0.0
50	BASIC	32.8% ± 4.1%	33.6 ± 2.1
50	ADVANCED	32.8% ± 4.1%	19.0 ± 4.1
50	AGGRESSIVE	32.8% ± 4.1%	19.0 ± 4.1
200	NONE	0% ± 0.0	200.0 ± 0.0
200	BASIC	34.2% ± 1.8%	131.6 ± 3.6
200	ADVANCED	34.2% ± 1.8%	74.0 ± 6.3
200	AGGRESSIVE	34.2% ± 1.8%	74.0 ± 6.3
1,000	NONE	0% ± 0.0	1,000.0 ± 0.0
1,000	BASIC	35.4% ± 1.6%	646.0 ± 16.4
1,000	ADVANCED	35.4% ± 1.6%	344.6 ± 31.6
1,000	AGGRESSIVE	35.4% ± 1.6%	344.6 ± 31.6

Simulated cache sweep (moved verbatim from the main text's Section 5.2; the main text retains the prose headline and the naive-arm comparison).

Table 18: Simulated API calls with and without the result cache (2,000 tasks per seed, 5 seeds, mean ± sample standard deviation; reduction equals hit rate by construction; the third, naive response-cache arm is reported in the text below).

Zipf exponent s	Calls (no cache)	Calls (cached)	Reduction = hit rate
0.5	2,000	810.4 ± 5.7	59.5% ± 0.3%
1.0	2,000	642.0 ± 3.8	67.9% ± 0.2%
1.5	2,000	408.0 ± 9.4	79.6% ± 0.5%

Paired per-pass decomposition (moved verbatim from the main text's Section 5.5; the main text retains the prose analysis).

Table 19: Paired per-pass decomposition on the Section 5.3 sequences (length 200, 5 seeds, identical seed base 2,000); the fusion + DCE row reproduces the Section 5.3 headline exactly because the inputs are identical.

Configuration	Instruction-count reduction
NONE	0.0% ± 0.0%
fusion only	21.6% ± 1.2%
DCE only	24.5% ± 2.6%
fusion + DCE	34.2% ± 1.8%

I.2.7 Chain-Enriched Optimization Workloads

Raw results: `examples/benchmarks/fusion_chain_results.json` (script `fusion_chain_bench.py`). **Purpose:** the Section 5.3 random-slot pools understate structure-sensitive passes; do chain-enriched workloads—shaped like the fusion/DCE/parallelism assumptions—amplify the BASIC → ADVANCED gain? **Generator:** segment families mixed at fusion_chain 0.35 / parallel_batch 0.30 / dataflow_chain 0.15 / dead_chain 0.08 / effectful_anchor 0.12, with per-family run-length ranges (fusion chains 4–12, parallel batches 4–10, dataflow chains 3–8, dead chains 2–6, effectful anchors 1), declared costs U(0.5, 2.0), declared memory sizes {64, 128} and dataflow buffers {256–2048}; the terminal sink consumes only leaf outputs. **Compiler:** the real `InstructionCompiler.compile` (IR captured around the call, no output patching); levels NONE/BASIC/ADVANCED/AGGRESSIVE (BASIC = DCE + fusion; ADVANCED adds parallel grouping and the resource plan; AGGRESSIVE adds the memory-optimization pass). **Design:** 4 levels × 5 seeds (0–4) × lengths {50, 200, 1000} × 2 arms = 120/120 cells complete; the control arm reuses the Section 5.3 generator `synthetic_formal_sequence` at the same seeds (`p_dead` 0.25, `p_pure` 0.55)—it reproduces the published Section 5.3 numbers at length 1000 (BASIC dispatch 646.0, ADVANCED 344.6, BASIC reduction 0.354), proving the gauge has not drifted.

Table 20: Chain-enriched ladder (mean ± sd over 5 seeds)

Length	Level	Effective count	Reduction	Dispatch units	Declared-cost makespan	Parallel width
50	NONE	50.0 ± 0.0	0.000	50.0 ± 0.0	61.6 ± 1.1	0.0

Length	Level	Effective count	Reduction	Dispatch units	Declared-cost makespan	Parallel width
50	BASIC	36.8 ± 7.3	0.264 ± 0.147	36.8 ± 7.3	59.8 ± 3.5	0.0
50	ADVANCED	36.8 ± 7.3	0.264 ± 0.147	5.6 ± 4.3	10.1 ± 7.0	28.2
50	AGGRESSIVE	36.8 ± 7.3	0.264 ± 0.147	5.6 ± 4.3	10.1 ± 7.0	28.2
200	NONE	200.0 ± 0.0	0.000	200.0 ± 0.0	247.9 ± 5.5	0.0
200	BASIC	137.6 ± 8.5	0.312 ± 0.042	137.6 ± 8.5	230.7 ± 6.2	0.0
200	ADVANCED	137.6 ± 8.5	0.312 ± 0.042	25.0 ± 8.5	43.6 ± 13.3	44.8
200	AGGRESSIVE	137.6 ± 8.5	0.312 ± 0.042	25.0 ± 8.5	43.6 ± 13.3	44.8
1000	NONE	1000.0 ± 0.0	0.000	1000.0 ± 0.0	1238.3 ± 12.2	0.0
1000	BASIC	705.0 ± 13.5	0.295 ± 0.014	705.0 ± 13.5	1172.7 ± 17.9	0.0
1000	ADVANCED	705.0 ± 13.5	0.295 ± 0.014	147.0 ± 29.2	242.5 ± 42.2	69.0
1000	AGGRESSIVE	705.0 ± 13.5	0.295 ± 0.014	147.0 ± 29.2	242.5 ± 42.2	69.0

Table 21: Random-slots control ladder (same seeds; the published Section 5.3 configuration)

Length	Level	Effective count	Reduction	Dispatch units	Declared-cost makespan	Parallel width
50	NONE	50.0 ± 0.0	0.000	50.0 ± 0.0	50.0 ± 0.0	0.0
50	BASIC	33.6 ± 2.1	0.328 ± 0.041	33.6 ± 2.1	33.6 ± 2.1	0.0
50	ADVANCED	33.6 ± 2.1	0.328 ± 0.041	19.0 ± 4.1	19.0 ± 4.1	7.0
50	AGGRESSIVE	33.6 ± 2.1	0.328 ± 0.041	19.0 ± 4.1	19.0 ± 4.1	7.0
200	NONE	200.0 ± 0.0	0.000	200.0 ± 0.0	200.0 ± 0.0	0.0
200	BASIC	131.6 ± 3.6	0.342 ± 0.018	131.6 ± 3.6	131.6 ± 3.6	0.0
200	ADVANCED	131.6 ± 3.6	0.342 ± 0.018	74.0 ± 6.3	74.0 ± 6.3	9.8
200	AGGRESSIVE	131.6 ± 3.6	0.342 ± 0.018	74.0 ± 6.3	74.0 ± 6.3	9.8
1000	NONE	1000.0 ± 0.0	0.000	1000.0 ± 0.0	1000.0 ± 0.0	0.0
1000	BASIC	646.0 ± 16.4	0.354 ± 0.016	646.0 ± 16.4	646.0 ± 16.4	0.0
1000	ADVANCED	646.0 ± 16.4	0.354 ± 0.016	344.6 ± 31.6	344.6 ± 31.6	14.4
1000	AGGRESSIVE	646.0 ± 16.4	0.354 ± 0.016	344.6 ± 31.6	344.6 ± 31.6	14.4

Findings. Chain enrichment amplifies the BASIC → ADVANCED dispatch reduction to −558.0 units (705.0 → 147.0, −79.1%) and the declared-cost makespan to −930.1 (1172.7 → 242.5, −79.3%) at length 1000, against −301.4 (646.0 → 344.6, −46.7%) on the random-slots control; the ADVANCED/BASIC dispatch ratio is 0.209 (chain) vs 0.533 (control), and peak parallel width 69.0 vs 14.4. The BASIC-level reduction is *lower* on chain-enriched pools (0.295 ± 0.014 vs 0.354 ± 0.016 control)—enrichment trades single-level shrink for structure the parallel pass can exploit. **AGGRESSIVE is structurally identical to ADVANCED** on all seven structural metrics, per seed, at every length and on both arms (deltas exactly 0.0); its only differentiation is the advisory memory plan, which is itself workload-dependent: buffer-reuse potential 107.8 ± 22.2 sites on chain-enriched length-1000 pools (11.7% of the 922.0 distinct tracked outputs; peak live outputs 814.2) vs 0.0 ± 0.0 on the random-slots control (549.4 tracked outputs, no reuse sites)—the highest optimization level's extra content exists only when chains supply the structure. **Limitations (as recorded in the results file):** the memory-optimization pass only annotates IR metadata—no executor consumes the plan, so memory figures are plan-derived analyses, not measured runtime savings; `declared_cost_makespan` is a metadata cost model (a parallel block contributes the max member cost), not wall-clock time; `peak_live_outputs` uses a conservative tie-break (production counted before release at equal positions).

I.2.8 Priced-Comparison Token-Mix Sweep

Raw results: `examples/benchmarks/promptcache_sweep_results.json` (script `promptcache_sweep.py`). **Purpose:** Appendix G's two caching strategies ranked differently on the real-gateway run (result cache ≫ prompt cache) and the mock paper-scale run (result cache still ahead, margin smaller); this sweep makes the token mix—an output of workload and harness ceilings—the explicit axis. **Design:** the Appendix G overlay ($3/M input, $15/M output, 0.1 × cache-read rate, same two simplifications, both favoring the provider baseline) applied to 13 mix points—the measured mock mix plus *modeled* constant completion lengths L ∈ {4, 8, 16, 24, 32, 48, 64, 128, 256, 384, 512, 1024} tokens/call (the observed reply-binds-at-max_tokens behavior of the Section 5.7.1 alias; L64 and L512 correspond to the max_tokens ceilings of the Section 5.7 cache run and the Section 5.7.1 harness)—× 3 Zipf skews {0.5, 1.0, 1.5} × 5 seeds (1000–1004) × 200 draws = 195 cells (39,000 draws) over the 40-task population, on the Section 5.7.1 workload shape (seven-step triage stream, one cacheable LLM-type step). All pricing and workload primitives are imported from `paper_real_llm_compiler_test.py`, not copied; the seed-1000 mock cells reproduce the published mock paper-scale artifact in all three pricing modes (anchor check `all_match = true`). All figures are MODELED-on-measured, mock-only.

Ordering invariance: `agentisa_cache ≤ provider_prompt_cache ≤ no_opt` in 195/195 cells, flipped 0 cells; the minimum per-cell gap is $0.016562 (MODELED). The Section 5.7.1 statement that the *ranking* depends on the token mix is therefore about the margin, and the margin is what the mix moves: the provider cache's effectiveness (its saving as a fraction of the result cache's saving) falls by a factor of 16.4 / 16.7 / 17.0 across skews from the input-dominated anchor to the completion-dominated one, crossing 0.5 near output-cost parity and collapsing below a 5% absolute saving well before the harness ceiling.

Crossover points (linear interpolation on the 5-seed mean curves): effectiveness crosses 0.5 at 23.3 / 22.9 / 22.5 completion tokens per call at s = 0.5 / 1.0 / 1.5 (output-cost share ≈ 44.4% / 44.4% / 44.3%); the provider prompt-cache saving falls below 5% beyond 392.3 / 397.7 / 416.7 completion tokens per call.

Table 22: Per-skew curves (5-seed means; "ratio" is provider-cache effectiveness, its saving ÷ the result cache's saving; "share" is the output-cost share of the no-optimization bill)

Mix point	s = 0.5: result / prompt saving %	ratio	share	s = 1.0: result / prompt saving %	ratio	share	s = 1.5: result / prompt saving %	ratio	share
L4	80.39 / 63.58	0.791	0.121	82.45 / 65.05	0.789	0.123	87.14 / 68.56	0.787	0.125
L8	80.40 / 56.70	0.705	0.216	82.47 / 57.93	0.702	0.219	87.19 / 60.94	0.699	0.222
L16	80.40 / 46.62	0.580	0.356	82.51 / 47.52	0.576	0.359	87.26 / 49.85	0.571	0.364
L24	80.40 / 39.58	0.492	0.453	82.54 / 40.29	0.488	0.457	87.31 / 42.18	0.483	0.462
L32	80.40 / 34.39	0.428	0.525	82.56 / 34.96	0.424	0.529	87.35 / 36.56	0.419	0.534

Mix point	s = 0.5: result / prompt saving %	ratio	share	s = 1.0: result / prompt saving %	ratio	share	s = 1.5: result / prompt saving %	ratio	share
mock (measured)	80.42 / 29.06	0.361	0.598	82.32 / 29.95	0.364	0.596	86.83 / 31.94	0.368	0.593
L48	80.40 / 27.24	0.339	0.624	82.59 / 27.65	0.335	0.627	87.41 / 28.86	0.330	0.632
L64	80.40 / 22.55	0.281	0.688	82.61 / 22.87	0.277	0.692	87.44 / 23.84	0.273	0.696
L128	80.40 / 13.36	0.166	0.815	82.65 / 13.52	0.164	0.818	87.51 / 14.06	0.161	0.821
L256	80.40 / 7.36	0.092	0.898	82.67 / 7.44	0.090	0.900	87.55 / 7.72	0.088	0.902
L384	80.40 / 5.08	0.063	0.930	82.68 / 5.13	0.062	0.931	87.56 / 5.32	0.061	0.932
L512	80.40 / 3.88	0.048	0.946	82.68 / 3.92	0.047	0.947	87.57 / 4.06	0.046	0.948
L1024	80.40 / 1.99	0.025	0.973	82.69 / 2.01	0.024	0.973	87.59 / 2.08	0.024	0.973

The result-cache saving is nearly mix-invariant (within-skew range across the whole 13-point axis: 80.39–80.42 at s = 0.5, 82.32–82.69 at s = 1.0, 86.83–87.59 at s = 1.5) and rises with skew, since it is determined by repeated structure; the provider prompt-cache saving is strongly mix-dependent (63.58% → 1.99% at s = 0.5). **Mix-coordinate alignment with the published real run:** the sweep's L512 point sits at output/input token ratio 3.53–3.66 and output-cost share 94.6–94.8%, against the published real-gateway run's 2.84 and 93.4% (quoted from its artifact, not recomputed: result cache cuts modeled cost 55.96%, provider prompt cache 3.28%, 180.36 prompt tokens and 512 completion tokens per call)—the modeled completion-dominated point is a reasonable analog of the real mix. **Limitations (as recorded in the results file):** mock only, no live gateway tokens in this sweep; the L* completion lengths are modeled constants, not generated text; a single workload shape; the overlay's two simplifications stand (no 1.25 × cache-write premium, no 1024-token minimum prefix—both favor the provider baseline); GPTCache and approximate caching remain out of scope, with no head-to-head claimed.

I.3 Real-Gateway Re-Runs: Quality Harness and Multi-Seed Cache Protocol

Four follow-ups to the Section 5.7/5.7.1 real-gateway experiments, run on the same gateway at the scales a per-call quota and a session budget allow. The first two use the original alias (a local alias for a hosted GLM reasoning model); the remaining three extend the same protocols to additional production model families (`claude-doubao-seed-2.0-code`, `claude-minimax-m3`, `claude-deepseek-v4-flash`) under a per-model-pool serialization discipline (concurrent load on one upstream alias was observed to starve parallel callers; runs on distinct aliases are independent). Both results files were recomputed from raw records by `examples/benchmarks/e10_gateway_postprocess.py` with a cross-check against the harness's own summary (both `passed`); raw per-call replies, stop reasons, token usage, and latencies are retained in the committed JSONs.

I.3.1 Classification-Quality Re-Run at a Raised Token Ceiling

Raw results: `examples/benchmarks/gateway_quality_results.json` (harness `examples/paper_real_llm_quality_check.py`, the Section 5.7.1 quality harness; raw run and probe files committed alongside).

Protocol. The Section 5.7.1 reference run bound 7 of 22 classification replies at the 1,500-token ceiling (empty text, the entire budget spent on reasoning), so its agreement numbers were fallback-dominated on those batches. This re-run raises the ceiling under an explicit probe-then-scale rule rather than guesswork: first one probe classification call at `max_tokens` = 4096 (15.25 s, `end_turn`, 719 completion tokens, 8/8 claims parsed—so the alias *can* finish a batch given budget); then worst-case planning from the minimum completion throughput observed in the 1,500-token run (37.6 tokens/s): a full-ceiling reply costs `max_tokens / throughput`, so 3072 tokens (2.05 × the 1500 default) plan to 81.6 s—just under the harness's hard-coded 90 s httpx timeout, above which a full-ceiling reply would trip the 3-strike run abort on exactly the batches that bind (the ceiling could not be raised further for this reason); at 81.6 s worst-case, the 40-minute session budget affords 29 calls, so the `tiny` preset (22 distinct batches, worst case 29.9 min) was chosen over `paper` (36 batches, 49.0 min worst case). Run configuration: tiny preset—50 draws over the 40-task population, Zipf 1.0, batch size 8, seed base 1000 (the same seeded draw sequence and batch set as the Section 5.7.1 reference run, enabling a paired comparison), one live call per distinct signature batch, temperature 0. Totals: 1 preflight + 22 live calls = 23 round trips, zero timeouts, zero crashes.

Table 23: Recomputed summary, with the 1,500-token reference run for paired contrast

Quantity	Reference run (max_tokens 1500)	Re-run (max_tokens 3072)
Distinct batches / live calls	22 / 22	22 / 22
Signatures	176	176
Parsed claims (parse rate)	120 (68.2%)	144 (81.8%)
Fallback-filled classifications	56	32
Replies bound at ceiling	7 / 22	4 / 22
Model claims vs gold	50.0% (120 claims)	51.4% (144 claims)
Final classification vs gold	43.2%	45.5%
Rule-fallback baseline vs gold	32.4%	32.4%
Final classification vs rule-fallback	48.9%	38.6%
Per-call latency (min / median / mean / max)	—	11.1 s / 29.3 s / 36.0 s / 81.9 s

Table 24: Per-gold-class agreement (planted rubric classes; 176 synthetic signatures)

Gold class	Signatures	Reference run matches	Re-run matches
critical	40	17 (42.5%)	19 (47.5%)
high	40	12 (30.0%)	12 (30.0%)
medium	51	19 (37.3%)	23 (45.1%)
low	45	28 (62.2%)	26 (57.8%)

Paired truncation outcome: of the 7 batches that bind at 1500, 4 now complete (including the highest-rank one) and 3 still bind at 3072 (empty text, the entire budget spent on reasoning);

1 batch that completed at 1500 bound at 3072 in this re-run (rank 11)—the reasoning alias is not bit-reproducible across runs even at temperature 0. **Verdict, reported as the harness issued it:** `all_pass` = False, solely because the `replies_not_bound_at_ceiling` gate fails on 4/22; every measurability precondition holds (one call per distinct batch, raw replies and gold labels recorded for every call, planted classes covered per batch, claims parsed). The harness gates no accuracy threshold—agreement rates are reported measurements. **What this does and does not show:** with the ceiling raised, the model's final classifications agree with the planted gold rubric at 45.5% vs the fallback rule's 32.4%—measurably above the rule, previously unknown from the truncated run—but 4 batches still cannot finish within 3072 tokens, so the model-quality statement carries a disclosed truncation residual rather than a clean all-clear. Gold labels are a planted keyword rubric on synthetic signatures (recorded verbatim in provenance), not human annotation of real logs; agreement-vs-gold is a rubric-following diagnostic, not a deployment-quality claim. Single gateway, single alias, single seed, sequential, no streaming: multi-model, multi-seed, and real-deployment behavior remain untested.

Trace-workload shape census (moved verbatim from the main text's Section 5.10). Shape census: 79 fan-out, 21 linear, 0 fan-in, 0 branch (7.9 steps/workflow); 152 dead-end steps exactly equal the planted guards; 106 of 692 adjacent pairs (15.3%) pass the compiler's fusibility gate, all CONTROL_FLOW. Disclosure: the corpus is generator-produced, not production logs; the census characterizes workload structure only.

I.3.2 Multi-Seed Real-Gateway Cache Run at Reduced Scale

Raw results in `examples/benchmarks/gateway_multiseed_results.json`:

- Bench: `examples/benchmarks/isa_real_llm_bench.py`—the Section 5.7 cache experiment's bench, run unmodified via CLI
- Raw per-Zipf chunk files: committed alongside the results JSON

Protocol. The Section 5.7 published run is 200 tasks × 1 seed × 3 Zipf levels; this re-run adds a second real seed at the reduced scale the session budget and gateway quota sanction: `--tasks 100 --seeds 2 --zipf 0.5 1.0 1.5 --max-tokens 64` (mirroring the Section 5.7 protocol's ceiling), seeds {0, 1} with `rng = random.Random(1000 + seed)`, tokens from the gateway's usage field and latency from the wall clock—no simulated values. **Quota episode, recorded as it happened:** the first single-invocation attempt crashed after roughly 125 successful calls with a gateway 429 `AccountQuotaExceeded` (a 5-hour usage quota, reset at 21:19:19 +0800); the bench does not retry 429s and writes no output file on crash. One 16-token probe call and five bench-shaped 64-token calls all returned 200 about ten minutes later, so the re-run was chunked per Zipf level with an outer 90 s × 3 retry wrapper instead of waiting for the reset; all 3 chunks passed on the first try with no recurrence. Totals: 1,103 gateway calls; chunked wall time 3,243.8 s (excluding the crashed attempt and inter-chunk gaps).

Table 25: Per-Zipf aggregates (2 seeds; expected hit rate is the analytic no-backend reference for the same seeded draw)

Zipf s	Calls (baseline → cached)	Hit rate	Expected hit rate	Wall saved (mean)	p50 cached latency
0.5	100 → 96.5 ± 3.5	3.5% ± 3.5%	3.5% ± 3.5%	12,655.9 ms	2,903.45 ms

Zipf s	Calls (baseline → cached)	Hit rate	Expected hit rate	Wall saved (mean)	p50 cached latency
1.0	100 → 84.5 ± 0.7	15.5% ± 0.7%	15.5% ± 0.7%	56,495.7 ms	2,675.25 ms
1.5	100 → 70.5 ± 3.5	29.5% ± 3.5%	29.5% ± 3.5%	89,201.8 ms	2,477.3 ms

Table 26: Raw per-(Zipf, seed) counts (all six cells; hit rate equals the expected reference exactly in every cell)

Zipf s	Seed	Calls (base → cached)	Hit rate = expected	Tokens saved	Wall saved (ms)
0.5	0	100 → 99	1.00% = 1.00%	85	12,456.2
0.5	1	100 → 94	6.00% = 6.00%	510	12,855.6
1.0	0	100 → 85	15.00% = 15.00%	1,275	38,894.5
1.0	1	100 → 84	16.00% = 16.00%	1,360	74,096.9
1.5	0	100 → 68	32.00% = 32.00%	2,720	90,372.1
1.5	1	100 → 73	27.00% = 27.00%	2,295	88,031.5

What this shows: the cache protocol's correctness property—measured hit rate equal to the analytic expected hit rate of the seeded draw—holds exactly on the real gateway in all 6 of 6 (Zipf, seed) cells, and call reduction matches hit rate at every skew (3.5% / 15.5% / 29.5%); per-call latency p50 falls mildly with skew (2,903 / 2,675 / 2,477 ms) as fewer uncached calls dominate. **What it does not show, stated plainly:** the reduced scale is *not* 1:1 comparable to the published 200-task run—100-task workloads contain fewer repeated (instruction, params) pairs, so the hit rates (3.5% / 15.5% / 29.5%) sit below the frozen paper values (8.5% / 26.0% / 45.5% at 200 tasks × 1 seed); the side-by-side is context, not a contradiction. Two seeds give a real but small stdev; the synthetic suite's 5-seed protocol would have cost about 2,760 gateway calls (the 1,103 recorded calls over six (Zipf, seed) cells extrapolated to the protocol's fifteen; each round trip ~2 s) and was not affordable in-session. Coverage: complete at reduced scale—single gateway, single model alias, 3 Zipf buckets × 2 seeds × 100 tasks, `max_tokens` 64.

I.3.3 Multi-Model Cache-Exactness Re-Run (`claude-doubao-seed-2.0-code`)

The frozen primary-alias run this section extends (moved verbatim from the main text when Figure 3 took over the visual): the GLM 5.3 flash cells under the identical protocol.

Table 27: Real-gateway cache behavior against the computed expectation (200 sequential requests, one seed, three Zipf levels, one model; paired cells read no cache -> cached).

Zipf s	Expected hit rate (computed, no backend)	Observed hit rate	Calls	Call reduction	Wall clock (ms)	Wall saved (ms)	Tokens saved
0.5	8.5%	8.5%	200 -> 183	8.5%	472,276 -> 405,754	66,522	1,445
1.0	26.0%	26.0%	200 -> 148	26.0%	455,946 -> 334,597	121,349	4,420
1.5	45.5%	45.5%	200 -> 109	45.5%	450,330 -> 244,316	206,014	7,735

Raw results: `examples/benchmarks/isa_real_llm_results_doubao_seed_2_0_code.json` (harness `examples/benchmarks/isa_real_llm_bench.py`, `--backend gateway`).

Protocol. Byte-identical protocol to Section 5.7: same workload generator (`zipf_workload`), instruction pool, parameter space, seed (0), N = 200 per Zipf level, `max_tokens` = 64, sequential httpx against the same gateway; the only differences are the model alias and the harness's hardened retry envelope used for this session (eight bounded retries at a 150 s per-call timeout, connection rotation across retries—`httpx.Limits(max_keepalive_connections=0)`—added after the gateway's load balancer was observed pinning retries to a sick upstream connection; defaults in the committed harness remain 3 attempts / 90 s). One preflight plus 6 × 200 = 1,200 measured round trips.

Zipf s	Expected hit rate	Observed hit rate	Calls	Cached-arm p50 (ms)	Wall clock (ms)	Tokens
0.5	8.5%	8.5%	200 -> 183	3998	868339 -> 808579	23811 -> 21769
1.0	26.0%	26.0%	200 -> 148	3732	807988 -> 593098	23735 -> 17526
1.5	45.5%	45.5%	200 -> 109	3743	729446 -> 422906	23606 -> 12868

Observed hit rates match the computed expectation exactly in 3/3 cells, as on the reasoning alias (Section 5.7). Per-call latency is higher on this model family (p50 3.6–4.0 s across cells versus 2.1 s on the reasoning alias), so wall-clock savings per avoided call are larger while the hit/miss statistics are unchanged—consistent with the exactness property (hit/miss decisions are a pure function of the (instruction, params) sequence and cannot depend on the backend).

I.3.4 Identity Re-Run with Live-Model Values, and a Model-Family Ceiling Census

Raw results: `examples/benchmarks/paper_real_llm_compiler_results_doubao_1500.json` and `examples/benchmarks/paper_real_llm_compiler_results_minimax.json` / `..._minimax_-1500.json` (harness `examples/paper_real_llm_compiler_test.py`, `--backend real --scale paper`).

Live-values run (`claude-doubao-seed-2.0-code`, `max_tokens` 1,500). Same harness, preset, seed, and cache protocol as Section 5.7.1, across all three Zipf cells. All 104 live calls (one per distinct signature batch, all by arm (a)) completed below the ceiling (max completion 142 tokens, zero bound) and replied in the requested `'<index>: <severity>'` format—verified once out-of-band by replaying a batch's prompt and running the committed `parse_classification` on the

reply (6/6 signatures resolved)—so the severity values entering arm (a)'s artifacts are the live model's own classifications, not the deterministic fallback. All mechanism checks re-passed on this model: `calls_match_expected`, arm (b) zero new gateway calls against the shared warmed cache, byte-identical artifacts across all realizations (verdict `byte_identical_all_realizations`), hot-path adoption, and the static 7 → 6 effective-instruction shrink.

Ceiling census (`claude-minimax-m3`). The same harness passes every mechanism check on a reasoning-style third model at two budgets, with replies binding at the ceiling in 101/104 calls (97%) at `max_tokens` 512 and 68/104 (65%) at 1,500. Combined with the reasoning alias (bound in every call at 64 and 512) and doubao-seed-2.0-code (zero bindings at 1,500), this locates ceiling-binding in the model family's verbosity under this prompt, not in the harness: the same protocol yields fallback-derived values on verbose reasoners and live-model values on terse completers.

Gateway quirk, recorded not pursued. `claude-deepseek-v4-flash` deterministically hangs on this harness's classify payloads (empty-message `ReadTimeout` across 19+ attempts spanning connection-rotated retries at both 512 and 1,500) while completing the quality harness's batched prompts over the same template at 1,500 with zero bindings (next subsection)—an upstream/alias-specific behavior, recorded as a deployment note; no run in this paper rests on it.

I.3.5 Quality on a Completing Model (`claude-deepseek-v4-flash`)

Raw results: `examples/benchmarks/gateway_quality_deepseek.json` (harness `examples/paper_real_llm_quality_check.py`, `--backend real --scale paper`).

Protocol. Identical harness, preset (200 draws over the 40-task population, Zipf 1.0, batch size 8, seed base 1000), gold rubric, and one-live-call-per-distinct-batch rule as the Section 5.7.1 quality diagnostic, with `max_tokens` = 1,500 and temperature 0; 36 distinct signature batches, one preflight, zero timeouts, zero crashes.

Quantity	Reasoning alias (Section 5.7.1, `max_tokens` 3072)	Completing model (`claude-deepseek-v4-flash`, `max_tokens` 1500)
Distinct batches / live calls	22 / 22	36 / 36
Signatures	176	288
Parsed claims (parse rate)	144 (81.8%)	288 (10000.0%)
Fallback-filled classifications	32	0
Replies bound at ceiling	4 / 22	0
Model claims vs gold	51.4%	5208.0%
Final classification vs gold	45.5%	5208.0%
Rule-fallback baseline vs gold	32.4%	3194.0%

Per-class agreement on the completing model: low 67.6% (74 signatures), medium 39.0% (77), high 47.8% (69), critical 54.4% (68). On this model every signature's final classification is the live model's own (zero fallback fills), and the model-over-rule margin is +20.1 points (versus +13.1 on the reasoning alias). The gold remains a planted keyword rubric over synthetic signatures, recorded verbatim in the results file; both runs are rubric-following diagnostics at one gateway, not deployment-quality claims.

I.4 Review-Motivated Control Experiments: Cache-Key Differentiation and Trace-Corpus Compile Control

Two deterministic, offline experiments added in response to external review; both artifacts carry full provenance (git rev, UTC timestamp, seeds) and are regenerated by single commands, with system-level smoke tests pinning their structural claims (`tests/system/test_review_control_experiments.py`).

I.4.1 I.4.1 Cache-Key Differentiation (Section 5.2's parity boundary)

Raw results: `examples/benchmarks/cache_key_differentiation_results.json` (runner `examples/benchmarks/cache_key_differentiation.py`).

Protocol. The Section 5.2 parity finding (instruction-level key vs. exact-response key, identical call counts) is a property of that benchmark's params, not of the mechanism. Two separating workloads, both using the production `build_cache_key` (no mock of the key function), over a Zipf-drawn task mix (s = 1.0, universe of 100 semantic params dicts of six keys, 2,000 draws, 5 seeds, seed bases 4,000/5,000): **W1** presents every task's params under a random insertion order—multi-key dicts whose semantic identity is order-independent—so the canonicalizing instruction key (`sort_keys=True`) hits while an order-sensitive response key (the serialized request string, as an HTTP cache sees it) misses on every reorder; **W2** alternates the template version (v1/v2, period 2) while params stay fixed, so the default params-only key false-hits (serving an entry cached under the other template—the Section 2.1 cache-validity condition violated by construction) while the strict namespace key (template folded in) misses exactly when the template changes.

Quantity	W1 reorder	W2 churn
Instruction/default key hit rate	95.0%	95.0%
Response key (insertion-order payload) / strict-namespace key hit rate	7.3%	90.4%
Order-canonicalizing response key hit rate	95.0% (gap 0.0)	—
Default-key false hits (W2)	—	48.3% of calls
Strict-key false hits	—	0

W1's gap (87.7 percentage points) is the parity-bound statement: an order-canonicalizing instruction key and an insertion-order response key diverge exactly when multi-key params are presented non-canonically. W2's 48.3% false-hit rate is what the strict namespace exists to prevent—correctness of the cache contract, not call reduction. Hit/miss accounting only; no LLM, tokens, or latency are involved, and the two arms of each comparison share one task stream. The runner's third key canonicalizes param order inside the response key itself; it matches the instruction key exactly (95.0%, 0.0 points, every seed), which locates W1's entire gap in canonicalization rather than instruction-level information—the instruction key's non-substitutable value is program-level reduction (Sections 5.3–5.5).

I.4.2 I.4.2 Trace-Corpus Compile Control (Section 5.3's non-planted-family bound)

Raw results: `examples/benchmarks/trace_compile_control_results.json` (runner `examples/benchmarks/trace_compile_control.py`).

Protocol. The Section 5.3 headline (34.2% effective-instruction reduction at length 200) is measured on linear sequences with optimization targets planted at disclosed rates. This control runs the same shipped `InstructionCompiler` at the same levels over the Section 5.10 corpus—100 generator-produced workflows, 792 steps, graph-shaped dependencies—where dead guards and fusible pairs are the generator's own, at its own rates. Deterministic full census; no seeds apply.

Quantity	Value
Workflows / steps	100 / 792
Dead steps (no outputs, no side effects)	152
Adjacent pairs passing the compiler's fusion gate	106
…of which both members survive DCE (live-live)	0
Effective instructions NONE → BASIC/AGGRESSIVE	792 → 640
Reduction vs. NONE (BASIC = AGGRESSIVE)	19.2%

The decomposition: 792 − 640 = 152 = the dead-step count exactly; all 106 fusions that fire absorb a dead guard (the corpus's pure transforms form dataflow chains, so no live pair passes the disjointness condition—correctly, since fusing a dataflow chain would change semantics). The 19.2% is therefore entirely dead-guard elimination, and the fusion share of the 34.2% headline is a property of the planted linear family, not of this corpus. The corpus is itself generator-produced (Section 5.10's disclosure applies): this experiment varies the workload family, it does not remove planting altogether.

I.5 Notation

Notation used across the main paper and this supplement, with the location of each symbol's definition. Section, definition, and theorem numbers follow the main paper; appendix references are to this supplement.

Symbol	Meaning	Defined at
τ_i	Per-call latency	Section 2.1
ω_i	Per-call context-reconstruction overhead	Section 2.1
δ_i	Per-instruction state-transition function	Appendix B.1
$C(n)$	Overhead of executing n function calls	Definition 2.1
$C_{cached}(n)$	Measured overhead under the result cache	Definition 2.1
$[\![P]\!]$	Denotation of program P	Appendix C.1
$\parallel$	Parallel composition (disjoint state footprints)	Definition 3.6 (Section 3.4.1)
$\circ$	Sequential composition	Definition 3.6 (Section 3.4.1)
$\otimes$	Capability-type composition	Section 6.3.1 (Appendix E.1)
cap	Capability type (Basic / Composite / Emergent)	Section 6.3.1 (Appendix E.1)
L	Security-level assignment on states and outputs	Section 6.3.6 (Appendix E.1)
s_k	Capability score of domain k	Definition 3.9 (Section 3.5)

Symbol	Meaning	Defined at
w_k	Capability-domain weight	Definition 3.9 (Section 3.5)
v_m, conf_m	Metric value / metric confidence	Section 3.5
$\phi_{A \to B}$	Cross-model instruction mapping $I_A \to I_B$	Section 6.3.2
$(I, T, \Sigma, \Delta, \Phi)$	The AgentISA core tuple: instruction set, type system, state space, transition-function set, verification-function set	Definition 3.1 (Section 3.1)
$pre(i)$, $post(i)$	Pre-/post-condition predicates of instruction i	Section 3.1.1 (Definition 3.5's obligation)
Π, Σ, μ	Type constructors: dependent function, dependent pair, recursive types	Section 3.1.2
s	Zipf skew parameter	Section 5.2
ϵ_i, δ_i	Per-mechanism differential-privacy parameters	Section 6.3.6

Symbol reuse. Σ is overloaded beyond the core tuple—state space (Definition 3.1) and the dependent-pair type constructor (Section 3.1.2); s denotes both the Zipf skew (Section 5.2) and arbitrary states in the semantics (Appendix B). δ_i is deliberately overloaded—per-instruction transition function (Appendix B.1) and per-mechanism DP parameter (Section 6.3.6), a reuse the main paper itself flags (Section 2.1); δ further denotes the state-machine transition function (Section 3.2.2) and the pipeline data-transfer function (Definition 3.7). τ doubles as the selection-pattern tree of Definition 3.8, and ϕ as the context-interpretation function of Theorem 3.3.

I.6 Second Real Family: Bash-Only Coding-Agent Trajectories

Section 5.12 measures tool-use traces in which read-only lookups dominate the stream. Coding agents have the opposite shape, so the same harness was run on a second recorded family.

Data. `nebius/SWE-agent-trajectories` (CC-BY-4.0, revision `68195a14`), shard `train-00000-of-00012`, the first attempt of each of the first 300 distinct instances—recorded SWE-agent runs on SWE-bench instances (swe-agent-llama-70b/8b/405b backbones), 8,502 actions in total, compacted by the same rule (keep ordered `[tool, args]`, drop prose and observations).

Mapping. The pilot's own conversion rules apply to both families and are the ones Section 5.12 cites: calls in the same turn are independent; each later turn consumes the immediately preceding step's result; unknown tools are effectful; and final-turn outputs are required, so liveness analysis cannot delete the answer.

Parsing, disclosed. These trajectories carry no structured tool calls: SWE-agent emits its action as the last fenced block of an assistant message and the next message is the observation. A block starting with `str_replace_editor` is an editor call (subcommand and path); anything else is a shell command whose first line is kept as `cmd`. This shard's backbones are bash-only: all 8,502 actions parse as shell commands and none as an editor call, so the family contains no read-only instruction at all.

	τ-bench (Section 5.12)	SWE-agent (I.6)
trajectories / distinct tasks	1,960 / 330	300 / 300

	τ-bench (Section 5.12)	SWE-agent (I.6)
tool calls	14,285	8,502
cacheable (side-effect-free) calls	1,789 (79% of the first-trial stream)	0 (0%)
verbatim repeats	4	4,009 (47.2%)
static reduction (BASIC, AGGRESSIVE)	0.0%, 0.0%	0.0%, 0.0%
cache hits within task / within run	1.73% / 41.70%	— (nothing cacheable)

Finding: repetition is not cacheability. A bash-only coding agent repeats 47.2% of its commands verbatim, yet every one of them is effectful, so the result cache has nothing it may serve—the cache's surface exists only where the stream is dominated by side-effect-free reads, as in the tool-use family. The static 0.0% also holds in both families, though for different structural reasons: sequential observation chaining in tool use (no provably dead step, no provably independent pair), and an effectful-only instruction mix in the bash agent.

I.7 Instruction Precipitation: Protocol, Curves, and Lifecycle

The value claim for instructionization is not that one trajectory can be compiled, but that instructions *sediment*: a repertoire built from earlier tasks serves later ones, so per-task work falls as experience accumulates. That claim runs through the implemented pipeline—trace mining, templates with parameter schemas, parameterized replay, an approval gate—but had never been measured on real traces. This appendix gives the protocol and the complete tables behind the paragraph in Section 5.12.

Streams. Precipitation is a property of a *coherent* workload, so streams are never formed by concatenating domains: retirement would fire on the domain switch and read as instability. The tool-use family therefore contributes its four source streams separately (gpt-4o-retail 115 tasks, sonnet-35-new-retail 115, gpt-4o-airline 50, sonnet-35-new-airline 50; first trial per task) plus a round-robin mixed stream over all of them. The coding family contributes its 300 tasks (first attempt per instance, I.6), split by backbone and also run mixed.

Template identity. A call instantiates a template: `(tool, parameter keys)` in *schema* mode for named-parameter tools, and `(tool, leading command tokens)` in *command* mode for shell calls whose payload is a command line.

Four rules, each mirroring shipped machinery. (i) *Promotion gate*: a template enters the repertoire after two occurrences, the default `AutoPromotionConfig.min_support`; until then it is a candidate, not a hit. (ii) *Credit rule*: a step counts as served only by a template promoted in an **earlier** task, so no task scores off instructions it just created. (iii) *Retirement*: a template idle for more than `max_idle` tasks is retired at a task boundary, the `deprecate_stale` policy; a retired template must be re-mined (two fresh occurrences) to return. (iv) *Serving* means parameterized replay of a known instruction—the path `test_replay_parameterization` covers—not composition into a larger instruction.

Tool-use family (max_idle = 40, min_support = 2).

stream	tasks	steps	served	coverage	quarters	created	active	retired	re-mined
gpt-4o-retail	115	794	762	95.97%	88/99/100/100%	15	13	3	1
sonnet-35-new-retail	115	872	834	95.64%	88/99/99/98%	18	14	8	6
gpt-4o-airline	50	347	320	92.22%	76/98/99/99%	13	13	0	0
sonnet-35-new-airline	50	327	299	91.44%	76/100/98/97%	14	13	1	0
mixed (round-robin)	330	2,340	2,245	95.94%	91/98/96/96%	26	12	57	43

Coding family (max_idle = 100, min_support = 2).

stream	tasks	steps	served	coverage	quarters	created	active	retired	re-mined
llama-70b	283	7,726	4,449	57.58%	43/54/63/67%	67	291	409	34
llama-8b	15	752	547	72.74%	51/89/53/52%	52	52	0	0
llama-405b	2	24	4	16.67%	9/0/23/0%	5	5	0	0
mixed (round-robin)	300	8,502	5,052	59.42%	55/52/60/67%	70	291	457	39

Two shapes of precipitation. With a small tool vocabulary the repertoire saturates almost immediately: twelve to fourteen templates serve 92–96% of the tool-use stream, and coverage is already 76–88% in the first quarter because the first few tasks introduce nearly every instruction the domain has. With a large, instance-specific command vocabulary the repertoire is deep and keeps growing: hundreds of templates serve 43% → 67%, and coverage rises monotonically on the dominant backbone. Reuse is also skewed—one instruction (`get_order_details`, shape `order_id`) serves 540 steps of the tool-use stream, while `bash` with the shapes `edit 1:1`, `python reproduce.py`, and `ls -F` serve 582, 393, and 192 steps of the coding stream. That skew is why a small active set can carry most of the traffic, and why retirement fires often while re-mining stays rare.

Boundary: recurrence is not instantiation, and I.9 measures the gap. Three-step windows recur in 75.0% of the tool-use stream's windows and 32.8–36.0% of the coding stream's, and it is tempting to read that as the compound-instruction opportunity. We do not: instantiating a compound from the window's first call requires binding later steps to *observation values*, which this corpus does not carry, so these figures remain recurrence upper bounds and are never added to coverage. Appendix I.9 repeats the count on a corpus that does keep observations: there, 17.3% of recurring compounds survive a replay test and carry 12.53% of dispatches, not 75.0% — recurrence overstates the opportunity roughly fourfold.

What the measurement does and does not mean. Served steps are identity-level reuse: the system recognizes an instruction it already holds and instantiates it with new parameters instead of treating the call as new work. This is consistent with the 0.0% static reduction of Section 5.12

rather than in tension with it—the repertoire's payoff is replay-shaped, not composition-shaped—and it is the machine-checkable core of the "instructions as an accumulating asset" claim: create on support, read by replay, update through the approval gate, retire on staleness.

I.8 Cross-Family Instruction Flow: Finite Action Spaces, Two Regimes

The precipitation experiment of I.7 runs on tool use and shell coding. Those are two points; a claim about instruction memory should not rest on two points, and the embodied framing makes the gap obvious: a household robot's actions are a closed vocabulary (go to, open, take, put, heat, …), so instruction memory there should behave like a database agent's, not like a coding agent's. This appendix adds three task families whose action spaces are *explicitly finite* and measures them with the same harness, same rules, same metric.

Data. `AgentGym/AgentTraj-L` (Apache-2.0), three environments: `alfworld` (household manipulation, 2,420 episode prefixes, 32,247 actions, 13 verbs), `babyai` (grid navigation, 810 records, 4,642 actions, 8 verbs), `sqlgym` (analytics, 3,000 records, 3,000 single-statement tasks, 1 verb). Actions are extracted from the last `Action:` marker of each assistant turn; entity numbers are stripped (`alarmclock 1` → `alarmclock`) so that a template is the operation plus its normalised arguments. Disclosed limitation: AgentGym ships per-step samples (an episode prefix and its next action), so these streams measure action-level precipitation across tasks, not the composition of a whole episode into one instruction; every action here is effectful, so the result cache has no surface in these families (as in I.6).

Results (min_support = 2, max_idle = 100).

family	action space	active templates	tasks	steps	served	quarters
household manipulation (alfworld)	13 verbs	103	2,420	32,247	84.94%	85.8/86.7/84.8/82.9%
grid navigation (babyai)	8 verbs	54	810	4,642	92.72%	89.0/93.4/93.8/94.8%
SQL analytics (sqlgym)	1 verb	16	3,000	3,000	45.17%	46.4/45.9/41.9/46.5%
tool use (I.7, retail)	22 tools	13	115	794	95.97%	88.1/99.1/100/99.4%
shell coding (I.7)	open	291	300	8,502	59.42%	43/54/63/67%

Two regimes, and why the second one matters. Where the *arguments* come from a closed vocabulary, coverage saturates immediately and stays high: household manipulation and grid navigation serve 85–93% of later steps from the first quarter on, because "go to cabinet", "open drawer", "move forward" are the whole repertoire and recombine endlessly. Where the arguments are open—SQL table sets, shell command lines—coverage either flattens at 45% (SQL: every query names a different table combination, so the verb is shared but the parameter is not) or climbs slowly as the long tail accumulates (coding: 43% → 67%). The two regimes are visible in the skew as well: `go to cabinet` serves 4,422 steps and `open cabinet` 3,434, while the most-used SQL template serves 38.

The law this supports. A finite action space is necessary but not sufficient for instruction memory to pay. What governs the payoff is *parameter reuse*: the same operation applied to arguments drawn from a small, recurring vocabulary. Software agents satisfy this for lookups and entity operations and fail it for free-form code; embodied agents satisfy it for furniture, receptacles, and objects; analytical agents satisfy the verb and fail the schema. This refines the embodied intuition rather than contradicting it—a robot's advantage is not that its verbs are few, but that its *objects* are few and recur.

I.9 Compound Instructions on Traces with Observations

I.7 refused to claim compound instructions: three-step windows recur in 75.0% of the tool-use stream's windows, but instantiating a compound requires that its later steps take their arguments from earlier *results*, and an inputs-only corpus cannot tell whether they do. This appendix supplies the missing half and measures what survives.

Data. The same τ-bench historical trajectories at revision `59a200c6d575`, re-read so that each call keeps the observation the environment returned for it (the `tool` message joined by `tool_call_id`): 325 tasks (first trial each), 2,275 calls, of which 2,221 (97.6%) have an observation. Observations are truncated at 400 characters; truncation can only hide a value from the binder, so every bindability figure below is conservative. The compactor is `examples/real_traces/prepare_tau_bench_observed.py`.

Definitions, all mechanical. An *atomic template* is a `(tool, argument keys)` signature, as in I.7. A *compound* is a contiguous k-gram of atomic templates (k = 2..4) with support 2. For an occurrence, each argument of each step after the first gets a *provenance*: `obs:i` if every scalar in the value appears in step i's observation (i < current; the latest match wins), `param` if it appears in the compound's entry arguments, `constant` if the value has no scalars, and `external` otherwise — a value the environment supplies from outside the call sequence, such as an order id the user typed. The *wiring signature* is the tuple of (step, argument, kind, source) for the whole occurrence. A compound is *signature-consistent* when every occurrence has the same signature; it is *replay-verified* when a spec learned on one occurrence (parameters abstracted, observation reads recorded as JSON paths) reproduces the arguments of every other occurrence exactly. *Earned reduction* substitutes verified compounds greedily, longest match first and without overlap, and credits k−1 dispatches per application; it is a dispatch bound from a binder, not an end-to-end latency or cost measurement.

Results (min_support = 2, max_k = 4).

k	recurring	signature-consistent	replay-verified
2	135	47	41
3	174	36	27
4	141	19	10
all	450	102 (22.7%)	78 (17.3%)

Substituting the signature-consistent set would absorb 15.03% of the 2,275 dispatches; substituting only the replay-verified set absorbs **12.53%** (285/2,275 steps, 178 applications) under a greedy longest-match policy that removes one dispatch per application and never overlaps two applications, so the figure is that policy's yield rather than a bound: a different tiling would absorb a different share. The difference is the price of assuming that identical signatures imply identical bindings:

24 compounds pass the signature test and fail replay, mostly because the observation shape moves between occurrences.

Where the binding actually comes from. Of 274 argument slots in verified compounds, 43 (15.7%) are read from an earlier observation or a constant, while 231 (84.3%) are external parameters. The compound level is therefore mostly *parameter plumbing*: fusing a chain saves the dispatches and the wiring decisions, but it does not derive most of its arguments from data — the values come from the user's request. This is the substantive answer to why an inputs-only corpus could not have produced the figure, and why the earned number is 12.5% rather than the 75.0% that recurrence suggested.

The strict test, for contrast. Requiring every later argument to come from an observation or the entry call — the strictest reading of "compiled" — passes only 7 compounds (1.6%) and absorbs 1.36% of dispatches. The two figures differ because they re-tile different compound sets under different definitions, not because of a per-compound marginal effect; we report 1.36% as the strict-definition yield and treat the gap as definitional.

Threats. Learning and verification happen within one corpus: a spec is learned on one occurrence and tested on held-out occurrences of the same family, which is a generalization test but not a cross-family transfer test. The 30 replay failures are reported rather than dropped, and they are the reason the mechanism refuses an occurrence whose observations no longer match the recorded read path instead of guessing a value. Environments are mock (τ-bench), so the reduction measures dispatch absorption on recorded traces, not production traffic. Cross-model transfer is the subject of I.11: parameter-only specifications transfer at 100%, observation-wired ones at 0%, which is why the path-based binding above is a same-model result.

I.10 External Baselines and Priced Effects

Section 5.12's 41.70% is produced by *our* cache: it serves only side-effect-free calls and keeps every key for a whole run. A reader is entitled to ask whether an ISA-agnostic exact-key cache would do as well, how much capacity the figure needs, and whether the reuse is dynamic or a fixed hot set. This appendix answers the part the corpus can answer, and states the part it cannot.

Protocol. 325 first-trial tasks (2,275 calls; 1,789 cacheable under the pilot's purity classification), key = `sha256(tool | canonical-args)`, three scopes (within a task, within a run, and global over the whole corpus). As an anchor, the emulation reproduces the paper's numbers exactly — 746/1,789 = 41.70% within a run, 31/1,789 = 1.73% within a task — which fixes that both use the same key, the same records and the same purity filter; a mismatch would void the comparison. The denominator is 1,789 of 2,275 first-trial calls: the 486 calls on state-changing tools are excluded by the purity mapping, and an unbounded cache shared across a run's tasks is assumed, so 41.70% is an upper bound for this policy. The committed run-scope split is 724 cross-task hits and 22 same-task hits, so reuse that is strictly cross-task is 724/1,789 = 40.5%.

scope	capacity	hit rate
within a task	unbounded	1.73%
within a run	8	25.54%
within a run	16	33.82%
within a run	32	34.82%
within a run	64	36.89%
within a run	128	38.62%

scope	capacity	hit rate
within a run	256	41.59%
within a run	512+	41.70%

Capacity buys the first 24 points, then saturates. An LRU of eight entries already reaches 25.5%; 256 entries reach 41.6%, and nothing more is available because the run holds no further repeat. The figure is therefore a property of the workload's key distribution, not of unbounded storage — good news for deployment, since the working set is small.

The reuse is not a fixed hot set. A single pooled top-N list built from the first halves scores 2.77% (N = 16), 5.20% (N = 64) and 13.63% (N = 256) on the second halves (866 calls); one list per run does better, 2.89%/5.89%/14.78%. An online LRU warm-started on the same first half scores **42.61%** on the same 866 calls (42.96% unbounded), so the gap is 29 points and eviction explains none of it (256 entries cost 0.35 points). The ceiling for any first-half-only policy is 14.78% — the share of second-half calls whose key was seen in the first half — and a per-run list at N = 256 reaches it exactly (128/866), so static selection is saturated at its ceiling; everything above it comes from keys first seen *during* the second half, which requires online promotion.

The pooled rate is not a run-level constant. The four runs differ: 48.95% (GPT-4o retail, 303/619), 16.28% (GPT-4o airline, 35/215), 50.21% (Sonnet retail, 352/701) and 22.05% (Sonnet airline, 56/254). Retail is 73.8% of the cacheable calls, so the call-weighted pooled 41.70% sits above the run-weighted mean of 34.37%; resampling the four runs gives a 95% interval of [19.2%, 49.6%]. The interval is wide because the corpus contains four runs, not because the measurement is noisy — it is deterministic. We report the pooled figure for comparability with Section 5.12 and the run-level range whenever a claim would otherwise read as a constant.

Same-turn repetition is unmeasurable here. Every one of the 2,275 assistant turns contains exactly one call, so the duplicate rate is a structural zero and the parallel-grouping lever of Section 3.4 cannot be exercised on this corpus. We report the zero as a scope limit, not as evidence about batching.

Path dependence is small. Every run starts from an empty cache, so the pooled rate is an average along the recorded order rather than a steady-state rate. The order turns out not to carry it: the first half of each run hits 40.69% and the second half 42.81%, the first task decile already sits at 43.96% and the last at 42.86% (deciles range 23.4–65.6%), and only 12.2% of all hits fall in the first decile. We therefore report 41.70% as a path average over the recorded order, which is what a deployed cache would also experience on an unmodified trace, not as the rate a reordering scheduler would see.

A global cache needs a decomposition, not a headline. Sharing one cache across all four runs gives 62.94% (1,126/1,789). That number is not reuse in the sense the paper claims, and the committed decomposition shows why. Per single backbone, the same global policy reaches only **40.53%** (GPT-4o, 338/834) and **42.72%** (Sonnet, 408/955) — indistinguishable from the per-run scope, because a single backbone contributes no cross-run traffic. The entire 21-point increment is a *task performed by both backbones*: the shipped key deliberately excludes model identity (Section 5.2), so those hits are legitimate for the cache, but they are one task counted twice, not two tasks sharing a key. The origin buckets are also order-dependent — assigning a hit to the stream that inserted the key first gives 14/371/354/387 in recorded order and 17/350/403/356 reversed — so we quote the per-backbone rates and treat the bucket split as a range, never as a constant.

Priced effects: one row is priceable, and it is not the cache. A cache hit replays a recorded tool result; the model turn that emitted the call is still in the trace, and no per-tool latency was measured. Converting the 746 hits into seconds or dollars at a *per-model-call* unit price (mean 2.30 s; 21 input and 64 output tokens) would be a unit error, so `cost_model_results.json` marks that row `convertible: false` and the paper quotes no figure for it. The two rows that *are* priceable both require an unmeasured assumption: a template match or a compound substitution removes the planner turn that would have emitted the dispatch, which was never instrumented, so their seconds, tokens and dollars are conditional bounds, zero without the assumption. At an assumed \$3/\$15 per million tokens those conditional bounds are 5,158 s and \$2.30 for 2,245 template-served steps, and 655 s and \$0.29 for 285 absorbed dispatches. Output tokens are censored at `max_tokens=64`, so every dollar figure is a floor; the mechanisms are never summed, because F2 and F3 count the same stream at different granularity.

What cannot be measured offline. A semantic (embedding) cache and a provider prefix cache both need inputs this corpus does not carry — prompts and embeddings — so no GPTCache-style or SGLang-style head-to-head is possible here, and none is claimed. The baselines above emulate standard caching policy, not those systems.

I.11 Cross-Model Compound Transfer

Section I.9 learns compound specifications on one corpus and verifies them on held-out occurrences of the same families. A harder question is whether a specification learned from one model's behavior still instantiates when a *different* model performs the same task. The corpus makes this testable: GPT-4o and Claude 3.5 Sonnet each ran the same retail and airline task sets, so a spec learned on one can be tested on the other with the tool vocabulary held fixed.

Protocol. Learn on source A, test on source B, in both directions, for retail and airline. A compound is *eligible* when its atomic templates are present in the learner; *exposed* when an occurrence exists in the tester; *transferred* when the learned spec reproduces every exposed occurrence's arguments exactly (the I.9 criterion). Intervals are 95% Wilson.

cell	eligible	exposed	transferred	rate over exposed	occurrence exact
GPT-4o to Sonnet, retail	49	36	33	91.7% [78.2, 97.1]	148/176 (84.1%)
Sonnet to GPT-4o, retail	23	17	17	100% [81.6, 100]	50/50 (100%)
GPT-4o to Sonnet, airline	32	18	18	100% [82.4, 100]	68/68 (100%)
Sonnet to GPT-4o, airline	8	5	5	100% [56.6, 100]	20/20 (100%)
within-source control (retail, half to half)	38	28	27	96.4% [82.3, 99.4]	100/102 (98.0%)

The unconditional rate. Measured over all 49 eligible compounds rather than the 36 the tester re-exposes, the retail cell transfers $33/49 = 67.3\%$; the 13 that do not transfer are compounds the second model never performed at all, which is itself a behavioural difference, not a specification error. Both denominators are reported because exposure is a post-treatment selection.

The wiring, not the vocabulary, is what a model change breaks. Splitting the same cells by how a spec obtains its arguments is decisive. *Parameter-only* specifications — those whose later steps take every argument from the compound's own parameters — transferred in 33/45, 17/23, 18/32 and 5/8 *eligible* cases (100% of the exposed ones in every cell, with occurrence-exact replay); the cases that did not transfer are ones the second model never performed at all. *Observation-wired* specifications — those that read a value out of an earlier step's result via a recorded JSON path — transferred 0 of 3 in the retail cell, the only cell where any is exposed, so no such specification survives; in the broader category that counts every specification merely *containing* an observation read (10/6/6/4 exposed across the cells) the transferred count is likewise zero. Individual occurrences do replay — 2 of 17 in the reverse retail cell — but never all occurrences of one specification. In the forward retail cell, 28 of 176 occurrence tests fail: 21 as path drift (the recorded path still exists but yields a different value) and 7 as argument mismatch. Zero slots are unbound, which rules out the trivial explanation that the tester lacks the tools.

Coverage, if the portable half were substituted. Parameter-only specifications cover 24.4% of the forward retail tester's steps (10.6% reverse), 34.9% of forward airline (12.8% reverse); observation-wired specifications cover 0.0% in every cell. Those are the same dispatch-absorption measure as I.9, so the cross-model penalty is stated in the paper's own units rather than in counts of specifications.

What that implies. Tool vocabulary is portable across models; a positional read into an observation is not, because the two models elicit different result shapes from the same mock environment. A compound instruction should therefore record an observation read as a *field selector* (+ retry rule) rather than a positional path, or re-derive its paths per backbone at promotion time. This is a design requirement that only the cross-model arm exposes — the within-family numbers of I.9 are consistent with either wiring.

Cross-domain exposure is zero by vocabulary, not by failure. Between retail and airline only four tools are shared (`calculate`, `get_user_details`, `think`, `transfer_to_human_agents`), one of 49 retail compounds is even expressible in the airline vocabulary, and none is exposed in its traces. We report this as a scope boundary rather than a transfer result. Task ids also collide across domains (50 collisions), so joining domains by task id would silently pair unrelated tasks — a trap we avoided by joining on task content instead.

Threats. Two models, two domains, one mock environment; the airline-reverse cell has $n = 5$ exposed compounds (8 eligible), so its 100% carries a wide interval. The retail 33/36 also pools nested compounds — 21 at $k = 2$, 10 at $k = 3$, 4 at $k = 4$ transferred (8 of 10 and 4 of 5) — that co-occur in the same tasks, so the Wilson interval treats them as independent trials and is narrower than the evidence supports; the per-arity cells are the honest granularity, and every non-transfer sits at $k >= 3$. Transfer is measured on recorded traces, not on fresh runs, so it shows that a spec *fits* a second model's behavior, not that it would improve that model's task success.